《轨迹大数据管理与分析》

Introduction to Data Science Frontier Technologies

高云君、陈璐、苗晓晔、张天明编著

高等教育出版社

目 录

第 3 章 轨迹大数据治理 1
3.1 轨迹大数据治理概述 1
3.2 轨迹质量评估 3
3.2.1. 轨迹质量评估基本概述 4
3.2.2. 通用数据质量评估方法 5
3.2.3. 轨迹专用质量评估框架 10
3.3 轨迹清洗 13
3.3.1. 轨迹清洗基本概述 13
3.3.2. 传统轨迹清洗 14
3.3.3. 基于深度学习的轨迹清洗 20
3.4 轨迹压缩 23
3.4.1. 轨迹压缩基本概述 23
3.4.2. 传统轨迹压缩 24
3.4.3. 基于深度学习的轨迹压缩 28
3.5 轨迹恢复 32
3.5.1. 轨迹恢复基本概述 32
3.5.2. 传统轨迹恢复 33
3.5.3. 基于深度学习的轨迹恢复方法 36
3.6 小结与展望 41
3.7 术语表 42
3.8 习题 44
参考文献 45
参考答案 49

第 3 章 轨迹大数据治理

随着移动互联网的发展，轨迹数据在规模迅速扩大的同时，其采集来源、生成环境和记录形态也日益复杂，为后续原始数据质量问题的产生埋下了隐患[1]。例如，卫星信号遮挡、定位设备误差和通信延迟可能造成轨迹点漂移、时间戳错乱或轨迹片段缺失；高频采样会带来大量冗余点，增加存储和计算压力；多源轨迹数据之间还可能存在坐标系统不统一、字段格式不一致和采样频率差异等问题[2]。如果不对这些问题进行系统治理，后续的轨迹挖掘、路径重构、行为识别和模型预测结果都可能受到严重影响。

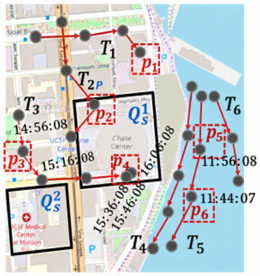
作为轨迹数据管理与分析流程中的基础环节，轨迹大数据治理是指通过一系列标准化、自动化的技术手段，对原始轨迹进行规范化的过程。本章围绕轨迹数据治理的基本概念、核心任务和典型方法展开，重点介绍轨迹质量评估、轨迹清洗、轨迹压缩和轨迹恢复等内容，为后续轨迹分析与应用奠定数据基础。

3.1 轨迹大数据治理概述

轨迹数据治理（Trajectory Data Governance）是对多源轨迹记录进行质量控制、标准化、组织与管理的过程。它在整个轨迹数据生命周期中扮演着承上启下的角色：向下屏蔽复杂的采集环境与底层设备误差，向上为城市计算、智能交通等高级应用提供坚实的数据底座。

作为一项综合性工作，轨迹数据治理涵盖的任务十分广泛，既包括数据质量提升，也包括数据组织、结构优化、语义增强、安全管理和应用适配等多个方面。在本章节中，主要选取轨迹质量评估、轨迹清洗、轨迹压缩和轨迹恢复四类具有代表性的基础任务进行介绍。它们分别对应轨迹数据治理过程中的质量诊断、错误修正、冗余削减和缺失补全等环节，是后续轨迹分析与应用的重要基础。

轨迹质量评估（Trajectory Data Quality Assessment）是轨迹数据治理中的前置环节，主要用于判断原始轨迹数据是否可靠、完整和可用。如图 3.1 所示，受采集环境与设备限制，原始轨迹常常会出现大量的质量缺陷，例如违反时间戳顺序（T_5），轨迹点丢失（T_3），轨迹漂移（T_2）等。通过轨迹质量评估，可以及时发现轨迹数据中的缺失、异常、漂移、采样不均和时空不连续等质量问题，并为后续的其他轨迹治理任务提供依据。因此，轨迹质量评估不仅是发现问题的过程，也是保障后续轨迹分析结果可靠性的基础。



Dimension	Trajectory	Description
Validity	T_5	p_5,p_6 *violate the timestamp monotonicity*
Completeness	T_3	p_3 *is a missing point*
Consistency	T_1	p_1 *is positioned incorrectly inside a building*
	T_2	p_2 *suffers from a location shift*
	T_3	*The vehicle should not stay on the road* (p_4)
	T_6	T_6 *deviates from the course of* T_4,T_5

图 3.1 轨迹质量评估任务的示意图

轨迹清洗（Trajectory Cleaning）主要用于处理原始轨迹数据中存在的噪声点、异常点、漂移点等问题，其目标是识别并修正错误观测，使轨迹数据更加可靠和稳定。如图 3.2 所示，相比于存在不合理局部跳变的原始轨迹（黑色曲线），经异常校正的修复轨迹（蓝色曲线）有效保留了原有变化趋势，且更加连续、稳定。在实际操作中，通常需结合距离、速度、加速度等时空与运动特征进行综合判断，对偏离正常规律的点位执行删除、修正或平滑操作。清洗结果应通过误差或下游任务指标验证，避免平滑处理掩盖真实运动变化。

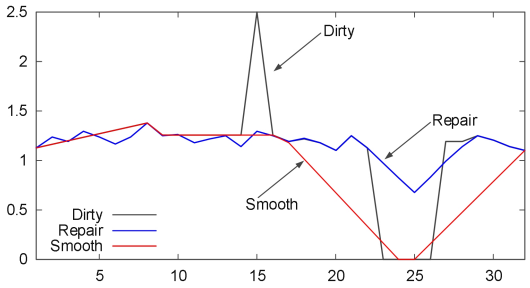


图 3.2 轨迹清洗任务的示意图

轨迹压缩（Trajectory Compression）旨在保留原始轨迹整体形态与语义特征的前提下，通过剔除冗余点来缩减数据规模。如图 3.3 所示，压缩过程并非简单的等距抽样，而是基于各点对轨迹形状的贡献度进行动态取舍：首尾节点（P_1、P_6）及体现方向变化的转折点（P_3）等关键信息通常会被保留；而对整体形态影响微弱的中间点（P_2、P_5）则作为冗余被删除。因此，轨迹压缩的核心是在降低存储与计算成本的同时，最大程度地避免关键运动特征（如转折、停留）的丢失，从而实现数据规模与信息保真度之间的平衡。

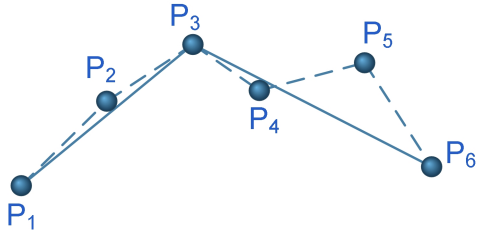


图 3.3 轨迹压缩任务的示意图

轨迹恢复（Trajectory Recovery）旨在解决由于采样稀疏或信号中断导致的片段缺失问

题，通过已有观测信息对不完整的行驶过程进行补全与重建。如图 3.4 所示，受限于实际观测条件（如摄像头盲区），车辆在路网中往往仅记录下离散的已观测点（红色节点），而未被捕捉的盲区则构成了轨迹断层（灰色节点）。轨迹恢复的核心，便是综合已观测点的时空关联、运动规律、路网拓扑及历史行为模式，借助插值、地图匹配或机器学习模型等方法，对缺失片段进行合理推断。通过这一环节，原本离散的中断轨迹得以还原为连续、完整的物理移动过程，全面提升了数据的最终可用性。

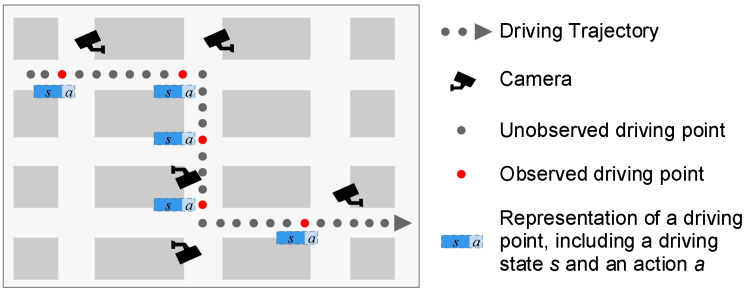


图 3.4 轨迹恢复任务的示意图

上述四类任务并不能涵盖轨迹数据治理的全部内容。在实际系统中，轨迹数据治理还可能涉及轨迹标准化、坐标转换、轨迹分段、地图匹配、语义标注、隐私保护、数据融合、索引构建和生命周期管理等任务。本章选取质量评估、清洗、压缩和恢复作为重点，是因为它们直接面向原始轨迹数据中最常见的质量与结构问题，能够较好体现轨迹大数据治理的基本思想和关键技术路径。

3.2 轨迹质量评估

在轨迹数据管理生命周期中，轨迹质量评估是保障后续分析、挖掘与智能决策可靠性的基础环节[3]。实际采集过程中，受定位精度、采样频率、信号遮挡和网络传输等因素影响，原始轨迹数据常包含误差、缺失、漂移和噪声。若未经评估便直接用于交通预测、路径规划、城市规划或行为分析等任务，可能导致分析结果偏差，甚至影响决策可靠性。

从数据治理角度看，质量评估主要发挥“诊断”作用，即在清洗、修复、异常检测或模型优化之前，先识别轨迹数据中存在的质量问题。与一般数据不同，轨迹数据不仅包含字段取值和时间顺序，还受到空间连续性、运动规律和地理环境约束。因此，仅依靠数值范围、字段完整性或简单逻辑规则，难以充分发现空间漂移、轨迹断裂和道路约束违反等问题。

因此，轨迹质量评估需要结合时空特征和应用需求，构建多维度评价指标。下面将围绕轨迹质量评估的基本概念、通用质量评估方法以及轨迹专用质量评估框架展开介绍。

3.2.1. 轨迹质量评估基本概述

轨迹质量评估是指对轨迹数据集的可用性、准确性、完整性和一致性等特征进行量化分析的过程。其核心目标在于发现数据中潜藏的错误、噪声和异常模式，并为后续的数据清洗与数据挖掘提供依据。例如，在轨迹校准、轨迹插补、停留点识别、路径重建和交通状态分析等任务之前，通常需要先判断轨迹数据是否满足基本质量要求。未经评估和验证的轨迹数据可能导致下游模型性能下降，甚至产生错误的分析结论。因此，轨迹质量评估是轨迹驱动型应用中的重要前置环节。

轨迹数据在实际收集和生成过程中，容易受到多种客观物理因素与技术瓶颈的干扰，从而产生不可避免的数据缺陷。这些质量问题的根源主要归结于以下三个方面。

定位系统存在测量误差：GPS、北斗卫星导航系统及移动基站定位均存在精度限制。在城市峡谷、高楼密集区等环境中，信号遮挡和多径效应会降低定位精度；在隧道、地下空间等区域，卫星定位还可能无法获得有效位置，使轨迹出现漂移或缺失。

采样频率受到资源约束：移动设备在采集轨迹数据时，需要在定位精度、存储容量和通信成本之间进行权衡。为降低能耗或减少通信开销，许多设备会采用较低的采样率。低采样率虽然可以节约资源，但会导致真实移动过程中的细节信息丢失，进而造成稀疏轨迹。

网络传输过程发生故障：轨迹数据通常需要从终端设备上传至服务器或云端。在网络信号不稳定、通信中断或数据传输延迟的情况下，部分轨迹点可能无法及时上传，甚至永久丢失，从而影响轨迹数据的连续性和实时性。

上述物理与技术层面的限制，会在轨迹数据的逻辑结构中表现为多种时空质量问题。根据其对轨迹时空特征的破坏形式，常见问题可归纳为以下几类。

空间与拓扑错误：定位漂移或坐标误差可能使轨迹点落入不合理区域。例如，道路车辆的轨迹点远离已知道路，可能提示定位或坐标转换问题。判断时需考虑定位误差容限、路网完整性及桥梁、隧道和停车场等场景，不能仅凭点位位于水域或建筑物范围内就认定为错误。

时间序列异常：同一移动对象的轨迹点通常应按照时间顺序排列，并满足时间戳单调递增的要求。如果由于设备时钟异常、数据乱序或多源数据合并错误，出现后续轨迹点时间戳早于前序轨迹点的情况，就会破坏轨迹的时间一致性。

连续性与完整性缺失：轨迹数据中可能存在采样间隔异常过长、部分时间段无记录、关键字段缺失等问题，导致轨迹在时间或空间维度上出现不连续现象。例如，连续两个轨迹点之间的时间间隔过大，可能说明中间存在大量未采集的轨迹点，从而造成轨迹片段缺失。

运动学逻辑冲突：移动对象的运动受到速度、加速度和方向变化等约束。若相邻点在短时间内发生不合理的位置跳变，或速度超过对象的物理上限，则相关观测可能异常。禁止停车区域的停留或偏离常见路线属于行为或语义异常，可能是真实事件，需与测量错误区分。

数据分布不均衡：轨迹数据在空间和时间上的覆盖往往并不均匀。例如，城市中心区域轨迹点密集，而郊区或偏远区域数据稀疏；工作日高峰时段数据充分，而夜间或节假日数据不足。这种时空分布不均可能导致下游模型在数据充足区域表现较好，而在数据稀疏区域产生较大的预测偏差或较高的漏检率。

由此可见，轨迹数据质量问题具有显著的时空交织性和复杂性。它们不仅涉及字段缺失、重复记录和异常取值等一般数据问题，还涉及空间拓扑、时间顺序、运动规律和分布公平性等轨迹特有约束。单一维度的数值校验规则难以满足轨迹数据质量管理的实际需求。因此，全面识别并量化这些缺陷，不仅是判断轨迹数据集可用性的必要手段，也为后续设计数据清洗算法、轨迹恢复方法和下游模型优化策略提供了基础。

3.2.2. 通用数据质量评估方法

通用数据质量评估方法可按是否考虑下游任务需求，分为任务无关质量评估（Task-agnostic Quality Assessment）[4,5]和任务感知质量评估（Task-aware Quality Assessment）[6]。前者关注数据本身是否准确、完整、一致和合理；后者关注质量问题对特定任务的影响。二者分别回答“这批数据本身是否可靠”和“这批数据是否适合当前任务”。

任务无关的数据质量评估不依赖具体下游应用，而是从轨迹数据自身的时空结构、记录状态和属性约束出发，对数据质量进行普适性判断。无论轨迹数据后续用于地图匹配、交通预测、行为识别还是出行模式挖掘，都需要首先满足基本的数据可靠性要求。因此，任务无关评估可以视为轨迹质量评估的基础层。

早期任务无关的轨迹质量评估主要依赖人工规则和物理约束，通常围绕空间质量、时间质量以及属性与逻辑质量展开，旨在衡量轨迹数据的准确性、完整性、一致性和逻辑合理性。

空间质量评估：空间质量主要用于衡量轨迹点坐标是否接近真实物理位置，以及轨迹点的空间分布是否合理。在缺乏真实位置基准的情况下，通常可以通过轨迹点是否落在合理活动区域内进行间接判断。例如，车辆轨迹点若大量出现在水域、建筑物内部或道路网络之外，往往说明数据中可能存在定位漂移或空间误差。同时，移动对象的运动应具有空间连续性，相邻轨迹点之间的距离不应超过其在对应时间间隔内可能达到的物理运动范围。若相邻两点距离过大，形成异常的“空间跳跃”，则通常可能存在空间质量问题。

时间质量评估：时间质量主要关注轨迹点时间戳是否正确、有序和连续。作为时空序列，同一轨迹内部的时间戳应满足单调递增关系，即$t_{i+1} > t_i$。因此，时间质量评估需要识别乱序记录、重复时间戳、时间回退等异常情况。这类问题通常来源于设备时钟不同步、网络传输延迟或多源数据合并错误。对于固定采样频率的数据，若相邻点的时间间隔明显增大，可能存在信号丢失或数据缺失；对于事件触发或自适应采样，应结合采集策略判断。

属性与逻辑质量评估：属性与逻辑质量主要检查轨迹附属字段是否完整、合理，并判断其是否符合基本物理和地理逻辑。首先，需要检查速度、方向角等关键字段是否缺失或异常。其次，需要验证属性与轨迹坐标之间是否一致，例如记录速度应与相邻轨迹点计算得到的运动速度大体一致，方向角变化应与轨迹整体走向相符。最后，还需要结合地理拓扑规则进行判断，例如车辆轨迹应符合道路连通关系和单行规则，高铁轨迹应依附于铁路网络等。通过这些检查，可以识别字段缺失、运动状态矛盾和拓扑约束违背等质量问题。

总体来看，早期基于规则的评估方法能够识别部分明显的轨迹质量问题，具有直观、易实现和可解释性强等优点。然而，这类方法往往是静态的、局部的和分散的，评估结果高度依赖人工经验和阈值设定。随着轨迹数据规模不断扩大、数据来源日益多样、处理流程更加复杂，仅依靠人工规则逐项检测已难以满足大规模轨迹数据质量管理需求。因此，研究者开始将分散的数据质量问题抽象为统一的指标体系，并进一步构建更加系统化、自动化和框架化的数据质量评估方法。下面将介绍一种代表性的任务无关质量评估方法，并分析其在轨迹数据场景中的应用方式。

TsQuality[7]是一种面向 Apache IoTDB 中大规模时间序列数据的质量评估系统。该系统将时间序列数据质量概括为完整性、一致性、及时性和有效性四个核心维度，并将相关指标实现为 IoTDB 函数或 Spark 算子，以支持大规模时间序列质量分析。

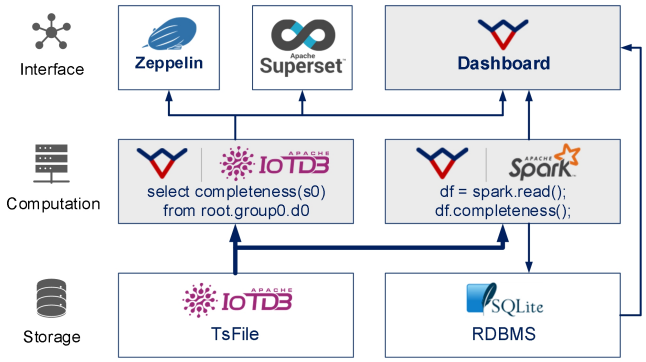


图 3.5 TsQuality 的框架图

TsQuality 的整体框架如图 3.5 所示，采用“存储层—计算层—接口层”的三层架构。存储

层以 Apache IoTDB 的 TsFile 作为时间序列数据的底层存储格式，同时使用 SQLite 保存质量统计结果。计算层提供两类质量评估方式：一类是将完整性、一致性、及时性和有效性等质量指标实现为 IoTDB 原生函数，使用户能够直接通过 SQL 语句查询某条时间序列或某一时间窗口内的数据质量；另一类是将这些指标实现为 Spark 算子，以支持大规模时间序列数据的批量质量计算。接口层则面向用户提供可视化和交互分析能力，包括 IoTDB 原生 Dashboard、Apache Zeppelin 和 Apache Superset 等工具。

由于轨迹数据本质上也是一种按照时间顺序组织的序列数据，其采样点通常由时间戳、空间位置及相关属性共同构成，因此可以借鉴 TsQuality 的指标体系，对轨迹数据质量进行更加系统的评估。

在轨迹数据场景中，可以借鉴 TsQuality 的四类维度，并按采集方式调整指标。完整性衡量预期采样点是否缺失；一致性检查重复记录及同一对象、同一时刻的位置冲突。这些问题可能影响采样频率、停留时间和运动特征计算。及时性衡量轨迹点的上传或入库延迟，适用于调度、交通监测等实时任务。有效性检查经纬度范围、速度和加速度等预设约束。路网可达性等检查是面向轨迹的扩展，不能视为 TsQuality 原生提供的全部功能。

由此可见，TsQuality 的主要贡献并不在于提出某一个具体异常检测规则，而在于将分散的时间序列质量问题组织为一套统一的度量体系。对于轨迹数据而言，采样缺失可归入完整性问题，重复轨迹点和冲突记录可归入一致性问题，延迟上传可归入及时性问题，速度越界、空间漂移和加速度异常可归入有效性问题。这种组织方式能够避免传统规则检测中指标零散、口径不统一的问题，使轨迹质量评估更加系统化和可解释。基于该体系，可以进一步按车辆、道路、时间段或空间区域生成质量画像，为后续轨迹清洗和建模提供依据。

传统的任务无关质量评估旨在提供一种通用的数据诊断标准，但在实际分析中，这种试图用单一标准适配所有场景的模式往往暴露出明显的局限性。为了弥补这一不足，任务感知的质量评估应运而生。任务感知的质量评估不仅关注轨迹数据本身是否存在缺失、重复、漂移或逻辑冲突，还进一步考虑这些质量问题对具体下游任务的影响。其基本观点是：数据质量并不是完全绝对的概念。同一种质量问题在不同应用场景下可能具有不同的重要性；同一份轨迹数据对于某些任务可能已经足够可用，但对于另一些任务则可能无法满足要求。

例如，短时间的轨迹缺失对于起讫点统计的影响可能较为有限，因为该类任务主要关注出发点、到达点和总体流向。然而，对于轨迹预测、驾驶行为识别或交通冲突检测而言，短时间缺失可能破坏连续运动模式，使模型难以准确捕捉速度、加速度和方向变化。又如，速

度字段缺失对于区域访问频次统计影响较小，但对于交通状态识别、拥堵传播分析和危险驾驶检测则可能造成显著影响。因此，任务感知评估并不只是判断“数据是否存在问题”，而是进一步判断“哪些问题会影响当前任务，以及影响程度有多大”。

从这一角度看，任务感知质量评估可以视为任务无关质量评估的进一步深化。任务无关评估给出数据本体层面的基础质量画像，而任务感知评估则根据任务目标对不同质量维度赋予不同权重，并据此判断数据是否适合当前应用。下面介绍两类具有代表性的任务感知质量评估方法，并分析如何将其应用于轨迹数据。

TFDV[8]是面向机器学习流水线的数据分析与验证工具，常用于检查训练数据、验证数据和线上服务数据之间的一致性。它通过数据统计信息和模式约束识别字段缺失、类型错误、异常取值及分布变化，并支持训练与服务偏斜和数据漂移检测。

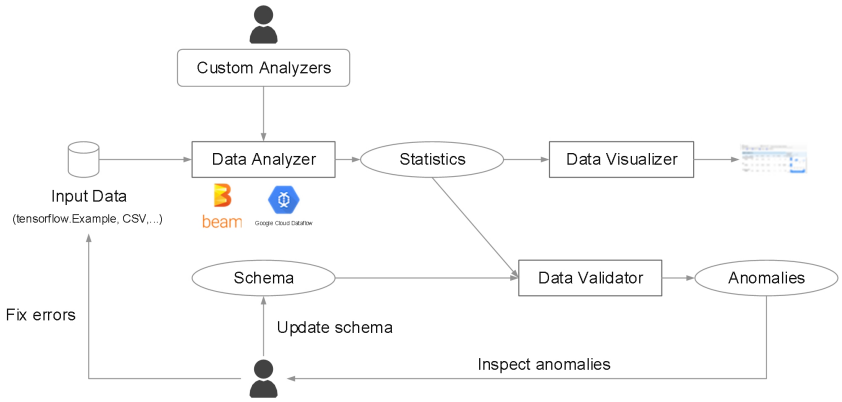


图 3.6 TFDV 的框架图

TFDV 的框架如图 3.6 所示，其核心流程包括数据统计、模式约束、异常检测和人工反馈四个环节。数据分析器生成输入数据的字段类型、缺失比例、取值范围等统计信息，并通过数据可视化器展示。随后，系统可据此推断初始模式，供用户审核与修订。数据验证器继续将新数据统计信息与模式及参照数据比较，并输出异常报告。用户据此修复数据或调整约束，而非让模式自动接受所有异常变化。通过这一闭环机制，TFDV 能够在模型训练或线上推理前及时发现数据质量风险。

在轨迹数据建模中，TFDV 的思想具有重要借鉴意义。例如，在轨迹预测任务中，模型训练阶段可能使用历史位置、速度、方向角、道路类型、时间周期等特征；而在线预测阶段，如果输入数据中某些字段缺失，或者速度、采样间隔、道路类型等特征分布与训练阶段明显不同，则模型性能可能下降。此时，数据质量问题不再仅仅表现为某个字段存在空值，而是表现为训练数据与线上数据之间的统计分布不一致。

对于交通预测系统而言，TFDV 类方法可以用于持续监控新进入轨迹数据与历史训练数

据之间的差异。例如，如果某一天某一区域的轨迹采样频率突然降低，或者某类设备上传的定位精度明显下降，系统可以及时发现数据分布变化，并提示模型可能需要重新训练或调整输入特征。因此，TFDV 主要提供统计验证与分布变化监测，并不直接量化质量问题对模型性能的影响，还需结合模型评估判断是否需要重新训练。

F4U[9]体现了另一类任务感知评估思想，即通过受控噪声实验评估数据质量变化对分析任务的影响。它向数据注入不同类型和程度的噪声，比较任务结果的变化，再估计质量因素与结果变化之间的关系。同一种缺陷对不同任务的影响可能不同，从而安排修复优先级。

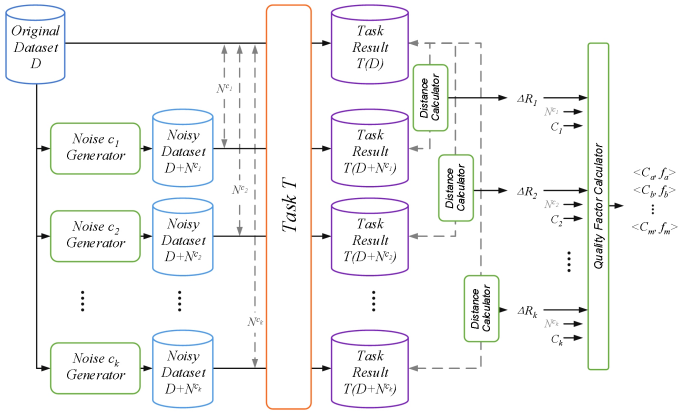


图 3.7 F4U 的框架图

如图 3.7 所示，F4U 首先在原始数据集上执行任务，得到基准结果；随后通过不同的噪声生成器，向数据中加入缺失、异常、扰动等质量问题，形成多组带噪数据集。接着，系统将同一任务分别作用于这些带噪数据，并比较其结果与原始任务结果之间的差异。差异越大，说明该类质量问题对当前任务影响越明显。最后，F4U 通过质量因子计算器分析噪声程度与任务结果变化之间的关系，从而判断不同质量问题的重要性。F4U 并不只关注数据本身“错了多少”，而是进一步关注这些错误会在多大程度上影响具体分析任务的结果。

在轨迹数据中，不同任务依赖的关键特征并不相同。对于轨迹预测任务，历史位置序列、速度、方向角和采样间隔通常是关键特征；对于交通状态识别任务，道路匹配结果、平均速度、车流密度和时间窗口覆盖率更加重要；对于异常行为检测任务，加速度、急转弯、急刹车和轨迹突变具有更强判别作用；对于路径推荐任务，起点、终点、道路连通性和历史路径选择则更为关键。因此，在不同任务中，同一质量问题的重要性会发生变化。

例如，若任务是实时交通状态识别，则速度字段缺失和速度异常应被赋予较高权重；若任务是离线出行模式挖掘，则轨迹完整性、停留点识别质量和起终点准确性可能更加重要；若任务是危险驾驶检测，则加速度异常、方向角突变和短时间空间跳跃应成为重点关注对象。

F4U 类方法的价值就在于帮助识别“哪些数据质量问题最影响当前任务”，从而指导数据清洗资源的优先分配。

任务感知质量评估通常可以表示为加权形式：

$$Q_{task}(D) = \sum_{k=1}^{m} w_k \, q_k(D) \tag{3.1}$$

其中，D表示待评估数据集，$q_k(D)$表示第k类质量得分，w_k表示该指标对当前任务的重要性权重，m 为指标数量。与任务无关评估不同，任务感知评估中的权重不是固定的，而是由具体任务目标决定。例如，对实时调度任务而言，及时性权重较高；对轨迹预测任务而言，时间连续性和运动学一致性权重较高。

综上，任务感知数据质量评估强调质量问题与任务目标之间的关联。它并不否定任务无关评估的基础作用，而是在其基础上进一步回答“哪些质量问题对当前任务最关键”。在实际轨迹数据治理中，通常应先通过任务无关评估识别数据的基础问题，再结合任务感知评估确定质量修复和数据筛选的优先级，从而在数据清洗成本与任务性能收益之间取得平衡。

3.2.3. 轨迹专用质量评估框架

前述方法均属于经典的数据质量评估方法，能够在一定程度上迁移到轨迹数据场景中。然而，这些方法本身并不是专门为轨迹质量评估设计的。具体而言，TsQuality 主要面向一般时间序列数据，TFDV 主要服务于机器学习流水线验证，F4U 则分析质量变化对任务结果的影响。它们虽然能够为轨迹质量评估提供有益借鉴，但并未充分考虑轨迹数据自身的特殊性，例如轨迹点之间的时空连续关系、移动对象的运动约束、道路网络等地理拓扑背景，以及轨迹数据在空间和时间上的分布偏差。因此，若直接将通用数据质量评估方法套用于轨迹数据，往往难以全面刻画轨迹数据中的典型质量问题。

为解决上述不足，研究者提出了专门面向轨迹数据的数据质量评估系统 T-Assess[10]。T-Assess 的核心目标是针对轨迹数据的时空序列特征、地理约束特征和应用需求，构建更加全面、高效和可扩展的轨迹质量评估框架。该系统认为，轨迹数据质量问题不仅表现为一般意义上的缺失值、重复值或异常值，还表现为空间位置错误、时间戳异常、轨迹断裂、轨迹偏移、道路网络约束违反以及时空分布不均衡等轨迹特有问题。

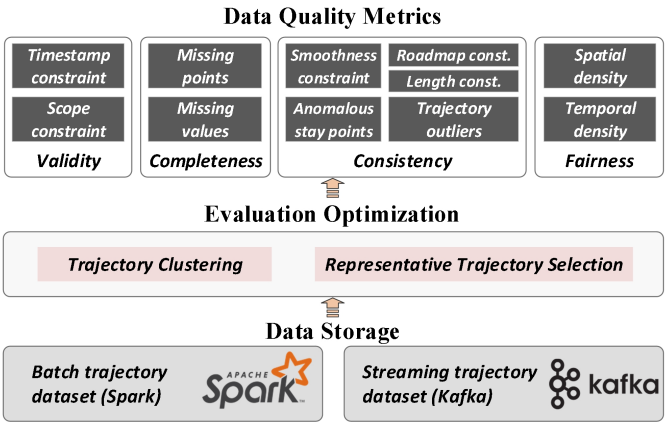


图 3.8 T-Assess 的框架图

如图 3.8 所示，T-Assess 从轨迹数据的实际错误类型出发，依托 Apache Spark 批处理引擎与 Apache Kafka 流式数据平台，引入轨迹聚类与代表性轨迹选择等优化策略，并在此基础上构建了涵盖有效性、完整性、一致性和公平性四个维度的质量评估体系。

从系统架构来看，T-Assess 主要由三层组成。底层是数据存储与处理层，用于支持不同形态的轨迹数据。对于历史轨迹数据，系统采用 Spark 处理批量轨迹数据集；对于实时产生的轨迹数据，系统采用 Kafka 支持流式轨迹数据接入。这种设计使 T-Assess 同时具备离线评估和在线评估能力。离线评估适用于历史轨迹数据质量分析、轨迹清洗前诊断和城市交通数据治理等场景；在线评估则适用于实时交通监测、在线异常检测和轨迹流质量监控等场景。

中间层为评估优化层，其核心思想是利用轨迹之间的空间相似性减少冗余计算。对于大规模轨迹数据，系统先聚类，再选择代表性轨迹进行检测，以近似估计整体质量。该策略可降低扫描成本，但空间路径相似不意味着设备误差、时间缺失或采样频率相同；其可靠性需要通过代表性选择和估计误差验证。

最上层为质量指标体系层。T-Assess 构建了面向轨迹数据的质量指标体系，主要包括有效性、完整性、一致性和公平性四个维度。有效性主要衡量轨迹数据是否满足基本的时空取值约束，包括时间戳约束和空间范围约束。例如，轨迹点的时间戳应按照移动过程单调递增，经纬度应处于合法范围内。如果时间戳顺序混乱，或者轨迹点落在不合理空间范围之外，则说明轨迹数据在有效性方面存在问题。

完整性关注轨迹数据是否充分记录了移动对象的运动过程。轨迹数据常因设备采样频率较低、信号丢失或传输中断而出现轨迹点缺失。当相邻轨迹点之间的时间间隔显著大于正常采样间隔时，系统可以判断该区间内可能存在缺失轨迹点。此外，如果轨迹点缺少经度、纬度或时间戳等必要字段，也属于完整性问题。完整性评估有助于发现轨迹断裂、轨迹稀疏和

位置字段缺失等问题，并为后续数据插补与轨迹恢复提供依据。

一致性是 T-Assess 中最能体现轨迹数据特殊性的维度。它关注轨迹点之间、轨迹与轨迹之间，以及轨迹与地理环境之间是否符合合理的运动规律和语义约束。具体而言，一致性指标包括平滑性约束、轨迹长度约束、轨迹离群检测、异常停留点检测等。平滑性约束用于识别轨迹中的异常位置跳变；轨迹长度约束用于检查轨迹是否满足预设长度要求；轨迹离群检测用于识别明显偏离正常路线的轨迹；异常停留点检测用于判断停留行为是否符合地理语义。例如，车辆轨迹点如果落入建筑物内部，或者轨迹路径明显偏离道路网络，需结合误差容限与地图完整性判断是否违反道路网络约束。

公平性从数据分布偏差的角度评估轨迹数据质量。轨迹数据在空间和时间上的分布通常并不均衡。例如，某些区域轨迹点密集，而另一些区域数据稀疏；某些时段采样充分，而另一些时段数据不足。这种分布不均衡会影响下游模型的学习效果，使模型在数据充足区域或高频时段表现较好，而在数据稀疏区域或低频时段表现较差。为此，T-Assess 通过空间密度和时间密度等指标刻画轨迹数据在空间区域和时间区间上的覆盖程度，从而提醒数据使用者关注潜在的数据偏差问题。

与前述通用数据质量评估方法相比，T-Assess 具有明显区别。TsQuality 提供了面向时间序列的质量度量体系，但并不特别关注道路网络、轨迹离群和异常停留点等轨迹特有问题；TFDV 关注机器学习流水线验证，F4U 关注质量变化对任务结果的影响，但并未形成面向轨迹时空约束的完整指标体系。相比之下，T-Assess 直接以轨迹数据为对象，从轨迹点、轨迹序列、地理拓扑和时空分布等多个层次刻画质量问题，更符合轨迹数据质量评估的实际需求。

从轨迹数据治理的角度看，T-Assess 的意义主要体现在三个方面。首先，它将轨迹质量问题从普通数据错误扩展为时空序列错误、运动逻辑错误、拓扑约束错误和分布偏差问题，丰富了轨迹质量评估的内涵。其次，它通过 Spark、Kafka、轨迹聚类和代表性轨迹选择等机制，提高了大规模轨迹数据质量评估的可扩展性，使轨迹质量评估能够适应批量历史数据和实时流数据。最后，它引入公平性维度，将轨迹数据质量与下游模型在不同区域、不同时段上的表现联系起来，为后续任务感知质量评估提供了重要基础。

T-Assess 代表了面向轨迹特征的专用质量评估框架。它表明，轨迹数据质量评估不能只停留在一般意义上的缺失值、重复值和异常值检测，而应充分考虑轨迹数据的时空连续性、运动规律、地理拓扑依赖、实时处理需求和时空分布偏差。只有将这些因素纳入统一评估体系，才能更加全面地判断轨迹数据是否真实、完整、合理，并是否适用于后续分析任务。

总体而言，轨迹质量评估的发展呈现出明显的层次化趋势。早期方法主要关注单点错误和简单规则违反，如坐标越界、时间乱序和字段缺失；随后，通用数据质量框架开始从完整性、一致性、及时性、有效性以及任务效用等角度提供更加系统的评估思想；进一步地，以 T-Assess 为代表的轨迹专用方法将质量评估与轨迹数据自身的时空属性、道路网络约束和分布覆盖问题结合起来，形成更加面向轨迹场景的综合质量评估体系。较为完整的轨迹质量评估框架应同时覆盖基础数据可靠性、时空运动合理性、地理拓扑一致性、分布公平性以及任务适配性等多个方面，从而为轨迹清洗、轨迹恢复、轨迹挖掘和轨迹智能分析等后续处理与分析任务提供可靠的评估基础。

3.3 轨迹清洗

在轨迹数据管理与分析流程中，轨迹清洗是继轨迹质量评估之后的重要处理环节。其核心任务是针对低质量轨迹数据进行修正、筛选与规范化处理，从而提升轨迹数据的真实性、连续性和可用性。由于轨迹数据具有显著的时空关联特征，单个轨迹点的异常不仅会影响该点自身记录的准确性，还可能破坏整条轨迹的运动逻辑，进而影响路径重构、停留点识别、行为分析和模型预测等后续任务。因此，在开展轨迹特征提取与应用分析之前，有必要对原始轨迹数据进行系统性的清洗处理。

与传统表格数据清洗不同，轨迹清洗不仅需要处理缺失值、重复值和格式错误等常规问题，还必须结合时间连续性、空间合理性以及移动对象的运动规律进行综合判断。轨迹清洗并不是简单地删除所有异常数据，而是在提升数据质量与保留真实运动特征之间寻求平衡。清洗规则和参数阈值应根据数据来源、采样频率、移动对象类型以及研究目标合理设定，从而为后续轨迹恢复、特征提取和行为分析提供可靠的数据基础。

3.3.1. 轨迹清洗基本概述

轨迹清洗是指针对原始轨迹数据中存在的错误、噪声、缺失、不一致和异常记录等问题，采用一定规则与方法进行识别、修正或剔除的过程。其核心目标在于提升轨迹数据质量，使其能够更加准确、连续地反映移动对象在时空维度上的真实运动状态。而实现这一目标的首要前提，便是准确识别并定义轨迹中的各类异常状态。

一般而言，异常轨迹是指在时间、空间、运动规律或地理环境等方面明显偏离合理状态的轨迹记录。它既可以表现为单个异常点，也可以表现为一段异常片段。例如，某一位置点由于定位误差突然偏离原有路径，可能形成孤立漂移点；连续两个位置之间的距离在极短时

间内发生大幅跳变，可能说明轨迹违反了移动对象的物理运动约束；车辆轨迹长时间偏离道路网络，或者落入建筑物、水域等不可通行区域，则可能说明其不符合地理拓扑约束。

相较于一般数据清洗，轨迹清洗的特殊性主要体现在时空约束和运动约束两个方面。一方面，轨迹点应满足时间顺序要求，即同一移动对象的轨迹记录应按时间顺序排列，并结合采集策略检查相邻点的时间间隔是否异常。另一方面，轨迹点还应满足空间合理性，即相邻位置之间的距离、速度和方向变化应符合移动对象的实际运动能力。

轨迹清洗并不是孤立的数据处理步骤，而是轨迹数据预处理流程中的基础环节，其结果将直接影响轨迹分段、地图匹配、停留点识别等下游任务。如果清洗不足，噪声与异常将被传递至后续分析过程，导致分析结果产生偏差；如果清洗过度，则可能误删具有实际意义的特殊行为，从而造成信息损失。因此，在实际应用中，应结合数据特性与应用场景合理制定清洗策略，在保证数据可靠性的同时，尽可能保留真实轨迹特征。

3.3.2. 传统轨迹清洗

传统轨迹清洗方法主要围绕“异常检测—异常修正—轨迹连续性保持”这一思路展开。由于轨迹数据同时具有一般数据问题和时空序列特征，相关方法通常从规则约束、信号平滑、几何统计和地图匹配等角度进行处理。其中，一般数据问题包括缺失、重复和噪声等；时空序列特征则体现在时间连续性、空间邻近性和运动约束等方面。

不同方法的侧重点有所差异。启发式方法强调基于经验规则快速识别明显异常；平滑滤波方法侧重削弱随机噪声；几何统计方法关注轨迹局部形态中的离群特征；地图匹配方法则借助道路网络修正偏离真实路径的轨迹点；混合式方法融合不同技术的优势以应对复杂的复合型噪声。下面对几类典型方法进行介绍。

启发式清洗算法是传统轨迹清洗中最直观、应用也较为广泛的一类方法。该类方法通常根据移动对象的物理运动规律或领域经验设定规则阈值，例如最大速度[11]、最大加速度[12]、最大空间跳跃距离和最小采样间隔等。当相邻轨迹点之间的速度或位移超过合理范围时，系统将相关点标记为待核查的异常候选，并进一步进行删除、修正或分段处理。

以 SCREEN 算法 [13]为代表，相关研究将速度约束引入轨迹清洗过程，通过判断相邻点之间是否满足预设速度范围来识别异常记录。这类方法的优势在于规则明确、实现简单、计算效率较高，适合用于大规模轨迹数据的初步筛查。但其局限也较为明显：清洗效果高度依赖阈值设定。阈值过宽可能漏检小幅异常，阈值过严则可能误删真实但特殊的移动行为。

算法 3.1 展示了流式场景下的 SCREEN 算法，该算法的核心思想是利用速度约束对当

前轨迹点进行局部修正，使清洗后的相邻轨迹点之间满足合理的速度范围。具体而言，对于当前轨迹点x_k，先以前一个清洗完成的点x_{k-1}^*为基准，计算其允许取值范围$[low, high]$（第 4-5 行），其中s_{min}和s_{max}分别表示最小速度和最大速度。然后，在滑动窗口w内，根据速度约束反推出当前点x_k的一个可能取值，并将这些可能取值加入候选集合C（第 6-9 行）。为了降低单个噪声点对修复结果的影响，算法取候选集合的中位数mid作为当前点的参考修复值（第 10 行）。若mid超出允许范围$[low, high]$，则将其修正到边界（第 11-14 行）；否则直接采用该值（第 16 行）。通过这种方式，SCREEN 算法既利用前一个已清洗点保证局部速度约束，又借助滑动窗口中的后续观测增强修复结果的稳定性，从而在满足运动合理性的同时尽量减少对原始轨迹形态的过度修改。

算法 3.1 流式场景下的 SCREEN 算法

输入： 待清洗数据集x，时间戳序列t，速度阈值s_{max}, s_{min}，滑动窗口 w
输出： 清洗完成的数据集x^*

1: $n \leftarrow length(\mathcal{T})$
2: **for** $k \leftarrow 1$ **to** n **do**
3: $C \leftarrow \{x_k\}$ // 初始化候选集合，保留当前原始值
4: $low \leftarrow x_{k-1}^* + s_{min} \times (t_k - t_{k-1})$ // 根据前一个已清洗点计算当前点的下界
5: $high \leftarrow x_{k-1}^* + s_{max} \times (t_k - t_{k-1})$ // 根据前一个已清洗点计算当前点的上界
6: **for** $i \leftarrow k+1$ **to** n **do**
7: **if** $t_i > t_k + w$ **then**
8: **break**
9: $C \leftarrow C \cup \{x_i + s_{max} \times (t_k - t_i)\} \cup \{x_i + s_{min} \times (t_k - t_i)\}$ // 生成候选修复值
10: mid $\leftarrow$ median(C) // 取候选集合的中位数作为参考修复值
11: **if** $mid < low$ **then**
12: $x^* \leftarrow low$ // 若低于允许范围，则修正为下界
13: **else if** $mid > high$ **then**
14: $x^* \leftarrow high$ // 若高于允许范围，则修正为上界
15: **else**
16: $x^* \leftarrow mid$
17: **return** x^*

平滑滤波清洗算法主要借鉴时间序列信号处理思想。该类方法认为，真实轨迹通常具有一定连续性，而由定位误差、信号遮挡或设备抖动产生的噪声往往表现为局部波动。因此，该类算法利用相邻轨迹点或历史观测值对当前位置进行估计，以削弱局部随机波动。常见的平滑滤波方法包括移动平均[14]、指数加权移动平均[15]、卡尔曼滤波[16]和自回归模型[17]等。

例如，卡尔曼滤波通过状态空间模型估计运动状态，将位置、速度等真实状态视为隐变量，将设备采集的定位点视为含噪观测，并交替执行预测与更新。利用运动学规律不断修正观测误差，从而提取出更接近真实的运动轨迹。

具体而言，设移动对象在时刻t的真实状态为x_t，观测到的轨迹点为z_t。卡尔曼滤波首先根据上一时刻的状态估计当前状态：

$$\hat{x}_{t|t-1} = F\hat{x}_{t-1|t-1} \tag{3.2}$$

其中，F表示状态转移矩阵，$\hat{x}_{t|t-1}$表示根据历史观测预测得到的当前状态。随后，算法将预测结果与当前观测值z_t进行比较，并根据二者的不确定性大小对预测值进行修正：

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t(z_t - H\hat{x}_{t|t-1}) \tag{3.3}$$

其中，H表示观测矩阵，K_t表示卡尔曼增益，$z_t - H\hat{x}_{t|t-1}$为观测残差。当观测值可信度较高时，算法会赋予观测值更大权重；当观测值偏离预测状态较大、可能受到噪声影响时，算法会更多依赖预测状态，从而降低异常观测对清洗结果的影响。

通过上述过程，卡尔曼滤波能够在保留轨迹整体运动趋势的同时，对局部抖动和随机漂移进行平滑修正。例如，车辆在道路上连续行驶时，若某一轨迹点因信号遮挡突然偏离道路附近位置，卡尔曼滤波会结合前一时刻的位置和速度预测当前位置，并对该异常观测进行削弱，使修正后的轨迹更加平稳连续。

然而，平滑滤波方法高度依赖于运动模型和噪声假设。当移动对象发生急转弯、突然停车或快速变速时，如果模型仍假设其保持平稳运动，可能会将真实行为误认为噪声并进行过度平滑。因此，在实际应用中，平滑滤波算法通常适合用于随机定位误差较明显、运动过程相对连续的轨迹清洗场景。

几何统计清洗方法[18,19]主要从轨迹点之间的空间结构和统计分布特征出发识别异常。轨迹可以看作由一系列时空点组成的线状结构，相邻点距离、方向角、转向角、局部曲率、速度变化率和邻域密度等指标均可用于判断轨迹是否合理。例如，若某一轨迹点与前后点形成明显尖峰，且该点明显偏离局部主运动方向，则可能是定位漂移造成的孤立异常点。

以 Akane[18]算法为例，该算法不是简单判断某个点是否超过某个阈值，而是从序列中挖掘“反复出现的局部模式”。如图 3.9 所示，它将数值序列离散化为符号序列，用马尔可夫模型刻画重复的局部模式，并在给定修改预算内寻找似然更高、困惑度更低的修正序列。因此，清洗过程可以理解为在尽量少修改原始数据的前提下，寻找一个困惑度更低、统计上更符合局部模式的序列。

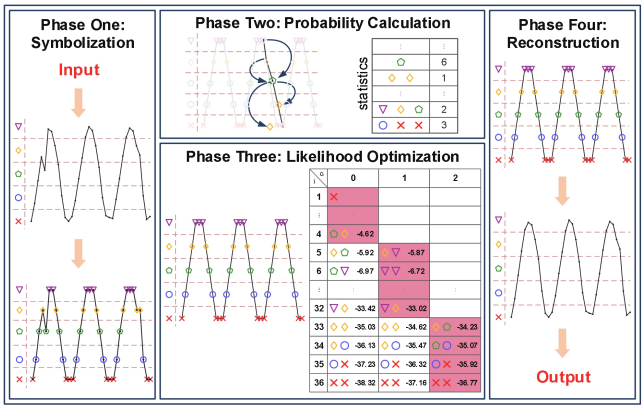


图 3.9 Akane 算法的框架图

具体而言，若将轨迹点的位置、速度、方向角、转向角或局部曲率等指标看作时间序列的特征列，Akane 的思想则可以用于识别那些与前后运动模式不协调的异常点。例如，在一段平稳前进的轨迹中，某个点突然偏离主运动方向，随后轨迹又回到原有趋势，这种局部结构在统计模式上较为罕见，会导致对应符号片段的概率下降。Akane 通过马尔可夫统计建模估计局部符号模式的出现概率，并在给定清洗预算下最大化修正后序列的似然，从而决定哪些点应被修改，而不是对所有可疑点机械地套用固定阈值。

与传统的启发式阈值算法相比，几何统计算法更强调上下文一致性和数据自身的统计规律。它不仅关注某个点是否“看起来异常”，还关注该点所在的局部片段是否符合历史中反复出现的运动模式。因此，这类方法可用于分析空间漂移、孤立跳点和连续异常片段。不过，当轨迹缺乏明显重复模式、采样较稀疏，统计模式可能不足以区分真实行为与异常噪声，仍需要结合速度约束、地图约束或业务规则共同判断。

路网匹配清洗算法的核心思想，是将轨迹数据中的异常点剔除与漂移修正，置于真实道路拓扑结构的强约束之下。对于车辆、公交、出租车等受道路网络限制的移动对象，其运动轨迹理应依附于物理路网；然而，受设备误差、信号遮挡或城市峡谷效应影响，GPS 观测点常出现严重偏离路面的漂移或跳跃现象。因此，该类方法并非在自由空间中盲目平滑数据，而是先为每个异常或存疑的 GPS 点召回若干候选路段，再综合空间距离、道路连通性、行驶方向及速度阈值等多维特征，鉴别出 GPS 点真正的物理映射位置，从而将偏离路网的脏数据强制纠偏或予以剔除。

以 TRMMA 算法[20] 的清洗流程为例，它巧妙地将局部误差修正建模为一个候选路段的分类问题。如图 3.10 所示，在处理包含噪声的轨迹序列时，算法首先圈定漂移点附近的一组局部候选路段，以控制计算开销。随后，结合 GPS 点的瞬时状态（如位置、航向）与轨迹的上下文特征，评估该点隶属于各候选路段的置信度。在确立各点的局部匹配结果后，算

法进一步利用道路网的最短路径和连通规则，对相邻轨迹点的合理性进行全局二次校验：若相邻点在真实路网中无法以合理的速度或时间抵达，则判定其中包含严重的逻辑冲突或离群噪声，进而触发过滤或二次修正机制。这种做法有效避免了传统“就近匹配”引发的修正谬误。

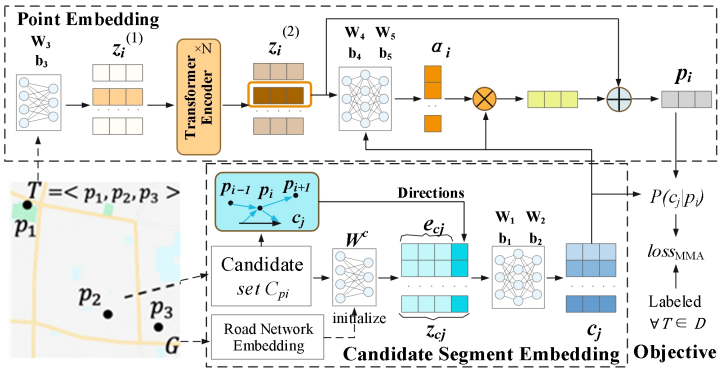


图 3.10 TRMMA 算法的框架图

路网匹配清洗算法的最大优势在于引入了“物理世界规则”作为清洗基准，使得清洗后的轨迹数据严格遵循交通逻辑。不过，该类方法的清洗质量高度依赖底层路网数据的精度，在面临复杂立交、密集平行辅路或路网信息缺失的区域时，算法可能因路网特征模糊而做出错误的纠偏决策，从而面临引入二次误差的风险。

最后，则是混合式清洗算法。混合式清洗算法是在单一清洗方法基础上发展起来的一类综合性方法[21,22,23]。该类方法结合多种清洗机制，在同一框架下利用规则约束、统计分布和多变量关系处理异常。与单一方法相比，混合式方法通常不再只判断某个轨迹点是否超过固定阈值，而是同时考虑该点是否满足物理约束、是否符合局部或全局统计规律、是否破坏轨迹的连续形态，以及是否与其他相关变量保持一致。这样既可以利用硬性规则排除明显不合理的数据，又可以借助统计模型发现较隐蔽的小幅异常，从而提升清洗结果的稳定性。

SHoTClean[23]是混合式清洗研究中代表性的算法，其基本思想可以概括为“以硬约束限定可行空间，以软约束选择最优路径”。 其中，硬约束（如速度、加速度）用于强制过滤不合理的物理连接，软约束（如偏离均值或局部模式的程度）则量化观测点的统计置信度。

与逐点判断异常不同，SHoTClean 将轨迹清洗转化为有向无环图（Directed Acyclic Graph，DAG）中的最大权重路径搜索问题。在该 DAG 中，节点代表原始轨迹点；若两节点在时间序列上相邻或跨度较小，且其派生的时空变量（如速度）满足硬约束，则建立有向边。边权重由软约束得分与时间衰减因子共同决定——衰减因子用于惩罚时间跨度过大的连接，保障轨迹的连续形态。最终，图中权重最大的路径所包含的节点被视为“正常轨迹序列”，而落选的点即被判定为异常脏点，并使用线性插值方法对脏点进行清洗修复。

如算法 3.2 所示，SHoTClean 的离线最大权重路径搜索流程如下：首先，算法为每个轨迹点计算软约束得分（第 1 行）；接着，利用速度上下界[V_(min),V_(max)]对点对之间的转移合理性进行硬约束校验，筛选出各节点的合法前驱节点集合（第 2-5 行）。为降低图搜索的计算复杂度，算法采用动态规划求解最大权重路径：依次遍历前驱节点，在状态转移时融入时间衰减因子γ，计算候选路径权重并更新最大累计得分 dp[i]及最优前驱指针 prev[i]（第 6-11 行）。最后，算法记录全局最大得分的终止节点与最优前驱（第 12-13 行），并通过回溯得到最大权重路径（第 14 行）。这种基于动态规划的求解范式极大降低了时间开销，能够高效应对长周期、大规模轨迹数据的清洗需求。

算法 3.2 SHoTClean 算法中离线场景下的最大权重路径搜索

输入： 待清洗数据集X，时间戳序列t，速度阈值V_{max}, V_{min}，偏离系数α，衰减因子γ
输出： 待清洗的脏点 dirty point

1: $s[i] \leftarrow \exp\left(-\alpha \cdot \left\| \frac{x_i - \mu}{\sigma} \right\|_2\right)$ // 通过均值和方差计算每个点的软约束得分
2: **for** $i \leftarrow 1$ **to** n **do**
3: **for** $j \leftarrow i-1$ **to** 1 **do**
4: **if** $\frac{\|x_i - x_j\|_2}{t_i - t_j} \in [V_{min}, V_{max}]$ **then** // 判断两点之间是否满足速度硬约束
5: $preds[i]$.append(j) // 将节点j加入到节点i的前驱节点中
6: **for** $i \leftarrow 1$ **to** n **do**
7: $dp[i] \leftarrow s[i],\ prev[i] \leftarrow i$ // 初始化路径得分和前驱节点
8: **foreach** $j \in preds[i]$ **do** // 遍历当前节点i的所有合法前驱节点j
9: $cand \leftarrow dp[j] + s[i] \cdot \gamma(i-j)$ // 计算从节点j转移到节点i的候选路径权重
10: **if** $cand > dp[i]$ **then**
11: $dp[i] \leftarrow cand, prev[i] \leftarrow j$ // 更新节点i的最大路权，并记录前驱节点j
12: **if** $dp[i] > best_score$ **then**
13: $best_score \leftarrow dp[i],\ e_id \leftarrow i$ // 更新最大路权，并记录终止节点
14: $Route \leftarrow$ backtrack_path$(prev, e_{id})$ // 通过反向回溯，得到最大权重路径$Route$
15: **return** dirty point $\{X_1, X_2, ..., X_n\} \setminus Route$ // 将不在$Route$中的点判定为脏点并返回

SHoTClean 的优势在于能够同时处理明显异常和隐蔽异常。对于大幅跳变、速度超限等问题，硬约束可以直接排除不合理连接；对于没有明显违反物理规则、但偏离正常轨迹模式的小幅漂移点，软约束得分也可以帮助算法识别其异常性。同时，该方法遵循最小修改原则，不会像平滑滤波方法那样对整条轨迹进行整体调整，而是尽量保留可信观测点，仅对被判定为异常的点进行处理，因此能够较好地保持原始轨迹的整体形态。

总体来看，混合式清洗算法是轨迹数据清洗的重要发展方向。它通过融合物理约束、统计规律、时间连续性和多变量关系，可弥补单一方法的部分局限。以 SHoTClean 为代表的

方法将硬约束与软约束统一到路径优化框架中，有助于分析明显跳变及部分较小偏移。未来，随着数据规模扩大和应用场景复杂化，混合式清洗算法仍需在参数自适应、实时处理效率和多源语义信息融合等方面进一步优化。

3.3.3. 基于深度学习的轨迹清洗

随着轨迹数据规模的持续扩大以及移动场景复杂性的不断提升，传统依赖人工阈值、统计假设或单一几何规则的轨迹清洗方法逐渐显现出一定局限。一方面，真实轨迹数据中的异常并不总是表现为明显的速度突变或空间跳跃，部分异常可能仅体现为局部轻微偏移、短时漂移，或多维特征之间的不一致；另一方面，轨迹数据通常具有复杂的时序依赖、空间关联和行为语义，仅依靠固定规则难以充分刻画其潜在变化规律。

因此，近年来深度学习方法逐渐被引入轨迹清洗任务。该类方法能够通过神经网络自动学习轨迹序列中的时空特征、上下文关系和潜在分布模式，从而实现异常点识别、缺失片段补全以及噪声轨迹修正。总体而言，深度学习下的轨迹清洗方法主要可以从序列建模、重构学习、注意力机制和生成式修复等方面展开。

循环神经网络（Recurrent Neural Network，RNN）及其变体长短期记忆网络（Long Short-Term Memory，LSTM）、门控循环单元（Gated Recurrent Unit，GRU）可用于轨迹序列建模。模型按时间顺序学习历史观测与当前状态的依赖，用预测误差识别异常候选，并可进一步估计缺失值或修复值。异常检测与数据修复需分别验证，较大预测误差也可能来自罕见但真实的行为。

以 GM-VSAE [24]方法（如图 3.11 所示）为例，该方法巧妙地将循环神经网络与概率生成机制相结合。与传统的“确定性单点预测”不同，该方法利用门控循环单元等结构实时编码历史轨迹的时空转移特征，并在每个时间步动态推断下一个轨迹点可能出现的空间概率分布，从而更好地包容复杂路况下的正常行为波动。

在实际的在线清洗任务中，当新的轨迹观测点输入系统时，模型会计算该点在已学到的正常模式下的“生成概率”。如果该观测点契合历史运动趋势与真实的交通通行逻辑，其生成概率将处于合理区间；反之，若该点的生成概率极低，系统便能实时将其甄别为离群脏数据并予以剔除。这种基于序列概率的评判机制，极大提升了清洗算法对隐蔽异常点的捕捉精度。

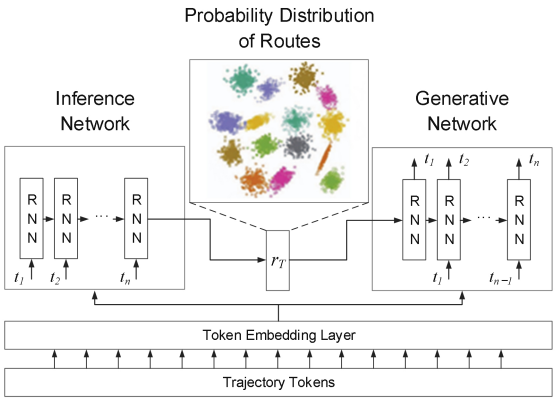


图 3.11 GM-VSAE 算法的框架图

然而，基于循环神经网络的清洗方法也存在一定不足。当轨迹序列较长时，模型可能难以充分捕捉远距离依赖关系；同时，模型训练通常依赖较大规模的高质量样本。如果训练数据本身包含较多噪声，模型可能学习到错误的运动模式，进而影响轨迹清洗结果的可靠性。

基于 Transformer 的轨迹清洗算法通常包括数据表示、序列建模、异常检测和轨迹恢复四个基本步骤。首先，在数据表示阶段，需要将原始轨迹点转化为模型可处理的向量形式。对于每个轨迹点，可以将其经纬度、时间间隔、速度、方向角和加速度等特征拼接为多维特征向量。由于 Transformer 本身不具备天然的时间顺序感知能力，因此还需要引入位置编码，用于表示轨迹点在序列中的先后顺序。这样，模型既能够获得轨迹点的属性信息，也能够感知轨迹点之间的时间位置关系。

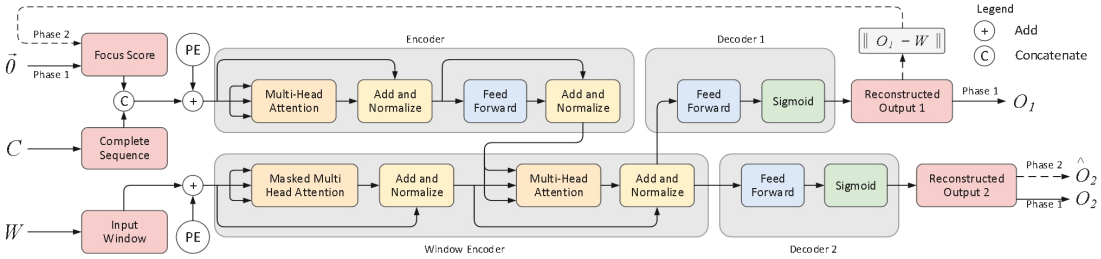


图 3.12 TranAD 的框架图

图 3.12 展示了 TranAD 模型[25]的框架。它利用 Transformer 编码—解码结构重构多变量时间序列，并由重构误差计算异常得分。输入窗口W描述当前局部观测，上下文序列C提供截至当前时刻的历史信息。通过这种设计，模型既能关注局部轨迹变化，也能利用全局时序模式辅助判断异常。

TranAD 包含序列编码器、窗口编码器和两个解码器，并采用两阶段重构。第一阶段生成初步重构结果O_1，由重构误差形成焦点得分；第二阶段将该得分作为自条件信息再次编码与解码，得到重构结果O_2。模型还通过对抗训练增强异常区分能力。最终以重构误差和阈值

判定异常，而非将所有重构困难的位置直接视为数据错误。

基于 Transformer 的方法能够较好地捕捉长距离时序依赖和复杂变量关系，适用于长轨迹、多维轨迹以及具有明显上下文关联的轨迹数据。但其不足也较为明显：模型训练成本较高，对数据规模和计算资源要求较大；同时，由于深度模型内部机制相对复杂，其清洗结果的可解释性通常弱于基于规则或统计假设的方法。

生成式模型主要通过学习轨迹数据的潜在分布，对缺失、异常或受噪声污染的轨迹片段进行生成和修复。常见结构包括生成对抗网络（Generative Adversarial Network，GAN）、变分自编码器（Variational Autoencoder，VAE）和扩散模型（Diffusion Model）。其基本思路是利用模型学习正常轨迹的分布特征，当输入轨迹存在噪声或缺失时，模型可以根据上下文生成更符合真实分布的轨迹点或轨迹片段。

ImDiffusion[26]是基于扩散插补的异常检测模型，其核心思想是将序列生成建模为一个逐步去噪的概率过程。不同于传统依赖局部特征平滑或固定物理阈值校验的清洗算法，扩散模型从全局分布重构的视角来攻克“去噪与纠偏”难题。如图 3.13 所示，模型在训练阶段先通过前向扩散过程，学习在高质量纯净序列中逐步注入噪声的退化路径；进而在反向生成阶段，掌握如何从高度无序的噪声分布中逐层“剥离”干扰信号，从而重建出符合真实地理与物理运动学规律的干净轨迹分布。这种深入理解真实序列潜在分布特征的建模方式，使得模型具备了在极高维度上识别异常并重塑受损时空数据的能力。

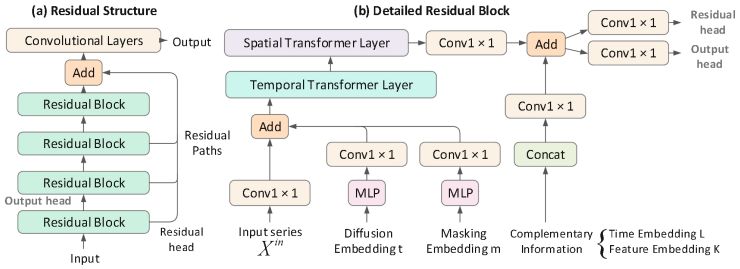


图 3.13 ImDiffusion 的框架图

然而，生成式方法也存在一定局限。该类方法往往更擅长发现和修复连续异常片段，对于孤立异常点的检测敏感性相对有限；同时，训练样本中的噪声也可能影响生成结果的可靠性。因此，生成式方法虽然在复杂轨迹补全和分布恢复方面具有较大潜力，但仍需要与物理约束、时空规则或外部地理信息相结合，才能进一步提高轨迹清洗的稳定性和可信度。

总体来看，深度学习方法为轨迹清洗提供了更强的非线性建模能力和自动特征学习能力。与传统方法相比，深度学习方法能够从大规模数据中学习复杂的时序模式、空间关联和多变量依赖关系，尤其适用于噪声类型复杂、轨迹规模庞大和移动行为多样化的应用场景。

然而，深度学习方法并不能完全替代传统清洗方法。一方面，其训练过程通常需要大量数据和较高计算成本；另一方面，模型结果的可解释性较弱，且容易受到训练数据质量的影响。如果训练数据中本身包含大量异常，模型可能将异常模式误认为正常规律。因此，在实际轨迹清洗中，更合理的做法是将深度学习方法与传统约束方法结合使用：利用速度、加速度、时间连续性和路网规则等硬约束保证清洗结果的基本合理性，同时利用深度模型学习轨迹数据中的复杂软约束和潜在模式。通过这种方式，可以在清洗准确性、模型鲁棒性和结果可解释性之间取得更好的平衡。

3.4 轨迹压缩

在轨迹数据管理与分析流程中，轨迹压缩是继轨迹清洗之后的又一重要环节。其核心任务是在尽可能保留轨迹主要形状、运动趋势和关键语义信息的前提下，删除冗余采样点以降低数据规模，从而有效减少数据存储、网络传输和计算分析的开销。随着移动定位设备和位置感知服务的广泛应用，轨迹数据通常具有采样频率高、数据规模大和时空连续性强等特点。若直接对原始轨迹进行存储与分析，不仅会增加系统负担，更可能降低轨迹相似性计算、模式挖掘和可视化分析等下游任务的效率。因此，在保证轨迹表达能力的基础上，对原始轨迹进行合理压缩具有重要意义。

与一般数据压缩不同，轨迹压缩并不是简单地减少数据字节数，而是需要结合轨迹点之间的时空关系和移动对象的运动特征，选择能够代表原始轨迹形态和变化趋势的关键点。这种基于点删除或低维表示的压缩通常是有损的，其目标并不是完全恢复原始轨迹，而是在压缩率和信息损失之间取得平衡。压缩方法和误差阈值应根据轨迹数据类型、采样频率、应用场景以及后续分析任务进行合理设定。在降低数据规模的同时，尽可能保持轨迹的时空语义信息，为后续下游任务提供高效可靠的数据基础。

3.4.1. 轨迹压缩基本概述

轨迹压缩是指在尽可能保留原始轨迹主要形状、运动趋势等关键语义信息的前提下，采用一定规则从原始轨迹点序列中选择关键点并删除冗余点，从而利用保留点近似表示原始运动过程的处理方法。其核心目标在于减少数据规模，降低存储、传输和计算分析成本，同时使压缩后的轨迹仍能较为准确地反映移动对象在时空维度上的主要运动特征。轨迹数据通常由带有时间戳和空间坐标的采样点按时间顺序排列而成，各点之间具有连续的时空关联关系。因此，轨迹压缩不仅要关注数据量的减少，还需要判断删除某些点是否会影响轨迹形状、

运动方向、速度变化及语义特征的表达。

从处理流程来看，轨迹压缩通常包括冗余点识别、关键点选择、误差度量、压缩结果生成及质量评价等环节。首先，需要根据空间偏差、时间同步误差、方向变化、运动状态或语义差异等因素判断轨迹点的重要性，从而识别对整体表达贡献较小的冗余采样点。其次，应选择能够反映轨迹转折、停留、速度或路径变化的关键点，并利用它们构成简化后的轨迹。此外，对于被删除的轨迹点，需通过垂直欧氏距离（Perpendicular Euclidean Distance，PED）、同步欧氏距离（Synchronized Euclidean Distance，SED）、方向误差或上下文误差等指标评估其造成的原始信息损失，以此判断压缩结果是否满足研究与应用需求。

相较于一般的数据压缩，轨迹压缩的特殊性主要体现在保持时空连续性和运动特征两个方面。一方面，压缩后的轨迹应尽可能保持原有的空间形态，即不应过度删除关键转折点、路径变化点和空间分布特征。另一方面，压缩后的轨迹还应尽可能保持移动对象的时间变化和运动规律，并通过误差指标检查时间顺序、速度和方向变化的偏差。例如，在车辆轨迹中，若删除了道路转弯处的关键点，可能导致压缩轨迹偏离真实的行驶路径。

此外，轨迹压缩并非孤立的数据处理步骤，而是预处理与高效管理中的重要环节。其结果将直接影响轨迹存储、查询、相似性计算、模式挖掘和可视化分析等下游任务。如果压缩不足，大量冗余的采样点仍会增加存储和计算负担，降低后续分析效率；如果压缩过度，则可能造成轨迹形状失真、运动细节丢失或语义信息缺失，从而影响分析结果的准确性。因此，在实际应用中，应结合数据规模、采样频率、移动对象类型和具体应用场景来合理制定压缩策略，确保在提高数据处理效率的同时，尽可能保留原始轨迹的关键时空特征和语义信息。

3.4.2. 传统轨迹压缩

代表性的传统轨迹压缩算法大体可分为三类：基于递归分裂的方法、基于逐步合并或点删除的方法，以及基于优化搜索的方法。前两类通常采用启发式或贪心策略，在效率和压缩质量之间进行折中；第三类则从全局优化角度寻找误差较小的简化轨迹，压缩质量较高但计算开销较大。

基于递归分裂的轨迹压缩是传统轨迹压缩中较为经典的一类方法。它通常采用自顶向下的处理思想：首先连接轨迹的起点和终点构成一条近似线段，并计算中间各点到该线段的偏差程度；若最大偏差超过给定阈值，则将该偏差对应的点作为新的关键点，并以此为分裂点将原轨迹划分为两个子轨迹，继续进行递归处理；若最大偏差未超过阈值，则直接用当前线段近似表示该轨迹片段。通过不断递归分裂，算法能够在删除冗余采样点的同时，尽可能保

留原始轨迹的整体几何形态。

以 Douglas–Peucker（DP）算法[27]为代表，这类研究将垂直距离误差引入压缩过程，通过判断中间点到起终点连线的最大偏差来选择关键点。DP 算法的具体过程是：首先连接原始轨迹的起点和终点形成近似线段，然后计算中间所有点到该线段的垂直距离。若最大距离不超过给定容差，说明该片段整体接近直线，仅保留起终点即可；若最大距离超过容差，说明存在较明显的形状变化，则需保留距离最大的点，并以其为界将轨迹划分为两个子片段继续递归压缩。该方法的优势在于思想直观、实现简单，能较好地保留轨迹的整体轮廓，尤其适合删除长直线路段上的冗余点。例如在车辆轨迹中，长直道路上的中间采样点对形态贡献较小，可直接删除；而在转弯、绕行或方向明显变化处，采样点通常会因偏差较大而被保留。

图 3.14 展示了基于 DP 算法的轨迹压缩结果。可以看出，处理后的原始轨迹采样点数量明显减少，且压缩轨迹能在一定程度上保持整体走向。然而，由于 DP 算法主要依据空间垂直距离选择关键点，其压缩效果对容差阈值十分敏感。若阈值设置过大，算法可能误删反映局部转向或细节变化的点，导致压缩结果与原始轨迹产生较大偏差；若阈值设置过小，又会保留过多采样点，难以达到理想的压缩率。因此，在实际应用中必须根据采样频率、移动对象类型及误差容忍范围来合理设定阈值。

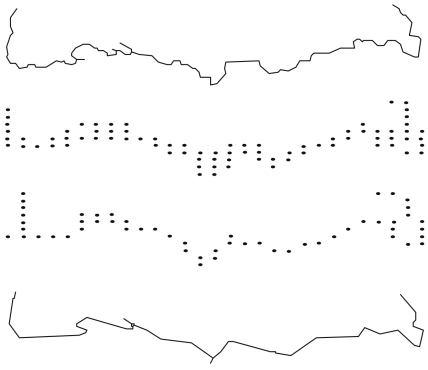


图 3.14 基于 Douglas-Peucker 算法压缩后的可视化轨迹

在 DP 算法的基础上，Hershberger-Snoeyink 算法[28]进一步对递归分裂过程进行了加速改进。该方法保留了 DP 算法“寻找最大偏差点并递归划分子轨迹”的基本思想，但通过引入更高效的数据结构与计算策略，有效减少了重复计算距离带来的开销。普通 DP 算法的最坏时间复杂度为 $O(n^2)$，而 Hershberger-Snoeyink 算法将最坏时间复杂度降低到了$O(n\log_2 n)$，从而大幅提升了大规模轨迹或线数据的处理效率。总体而言，递归分裂类方法具有较好的可解释性与较强的几何形态保持能力，但其压缩质量高度依赖于误差阈值和距离度量的选择，

且对时间信息、速度变化及语义特征的保持能力相对有限。

基于逐步合并的轨迹压缩是传统轨迹压缩中的另一类常见方法。与递归分裂自顶向下的划分方式不同，该类方法通常采用自底向上或局部删除的思想，通过评估相邻片段合并或单个点删除后引入的误差，逐步减少轨迹点数量。其基本思路是优先合并误差较小的片段，或优先删除对整体形态影响较小的采样点，从而在降低数据规模的同时，尽可能保持轨迹的主要形状与运动变化。代表性算法包括 Bottom-Up 方法[29]和 SQUISH[30]等，其中 Bottom-Up 多用于离线轨迹简化，SQUISH 则更适用于在线轨迹压缩场景。

Bottom-Up 是一种典型的基于逐步合并的压缩方法。它通常先将原始轨迹划分为由相邻点构成的短线段或局部片段，然后计算相邻片段合并后产生的误差，并优先合并误差较小者。随着合并的不断进行，采样点数量逐步减少，直到继续合并会超过给定误差阈值或达到预设的压缩比例为止。如图 3.15 所示，一条包含 10 个点的原始轨迹 $\langle p_1, p_2, \cdots, p_{10}\rangle$经过 Bottom-Up 处理后，可简化为仅含 4 个关键点的压缩轨迹 $\langle p_1, p_3, p_6, p_{10}\rangle$。该方法的优势在于思想直观、实现简单，能通过局部合并逐步删除冗余点。例如在出租车分析中，若车辆在某路段保持稳定的行驶方向，相邻片段合并产生的误差较小，中间点即可被删除；而在转弯、绕行或急剧变向处，合并误差通常较大，相应点更可能被保留。然而，Bottom-Up 主要依赖局部合并决策，压缩效果受误差度量和阈值设置的影响较大，往往难以保证全局最优。

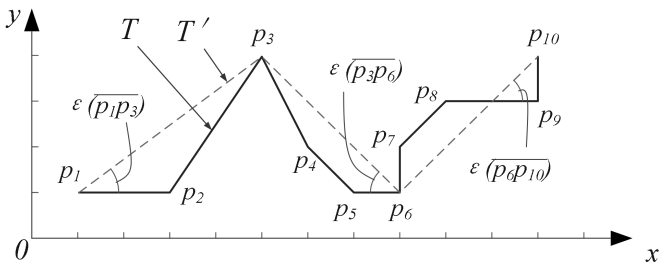


图 3.15 基于 bottom-up 的轨迹压缩

SQUISH 则是一种面向 GPS 轨迹流的在线压缩方法。Bottom-Up 通常需要在完整轨迹已知的情况下进行离线处理，而 SQUISH 能够随着轨迹点的不断到达实时完成压缩。该方法通过按压缩预算设定的缓冲区β来控制可保留的轨迹点数量，并为缓冲区中的每个点计算重要性值。具体而言，当新的 GPS 点到达时，算法先将其加入缓冲区；若容量超过限制，则计算删除各候选点可能引入的同步欧氏距离误差，并优先删除重要性最低的内部点，再更新相邻点的误差优先级；首尾点予以保留。通过这种方式，SQUISH 能够在有限的存储空间内持续维护一条简化轨迹。

SQUISH 的优势在于能够在线处理数据，无需等待完整轨迹采集完毕，非常适用于移动设备定位、车辆监控和实时传输等场景。例如在出租车轨迹采集中，若车辆长时间沿直线或近似匀速行驶，中间点对形态和速度变化的贡献较小，可被优先删除；而在急转弯、速度突变或路径明显变化处，相关点通常重要性较高，更容易被保留。不过，SQUISH 的压缩效果受缓冲区大小和局部误差估计的影响较大。当缓冲区过小或压缩比例过高时，局部删除带来的误差可能会不断累积，导致压缩结果与原始轨迹之间的同步位置误差逐渐增大。

基于优化搜索的轨迹压缩是传统方法中以全局误差优化为目标的一类方法。与递归分裂和逐步合并主要依赖启发式规则或局部决策不同，优化搜索类方法通常将压缩过程建模为分段逼近、关键点选择或误差最小化问题，并通过动态规划、搜索策略或可行性判断等方式寻找满足约束的较优结果。其核心思想是在给定保留点数、存储预算或误差约束的条件下，从原始轨迹中选择一组关键点，使压缩前后轨迹间的总体误差或最大误差尽可能小。代表性算法包括 Bellman[31]和 Error-Search[32]等。

Bellman 算法是一种典型的基于动态规划的曲线近似方法。它将轨迹简化看作分段线性逼近问题，即用若干条线段近似表示由连续采样点构成的原始轨迹，并通过优化分段端点的位置，使总体逼近误差尽可能小。具体而言，该方法首先将整条轨迹划分为若干候选子区间，每个子区间由一条线段或低阶函数进行近似表示；随后利用最优性原理，将全局误差最小化问题分解为多个子问题递推求解，从而得到给定保留点数或误差约束下的较优分段结果。

该方法的优势在于能从全局角度选择分段点，避免完全依赖局部贪心决策所带来的误差累积问题。因此，在轨迹简化中，它能根据整体误差分布选择更合适的关键点，在保留较少采样点的同时尽可能减小整体逼近误差。例如，对于包含多个转弯或局部波动的车辆轨迹，动态规划能综合比较不同分段方案的误差大小，从而选择整体效果较优的结果。但 Bellman 的计算开销通常较高，当轨迹点数量较大或候选分段较多时，求解过程会变得复杂，因而更适合离线或中小规模的数据处理场景。

Error-Search 则是一种面向方向误差最小化的压缩方法。与 Bellman 主要关注整体逼近误差不同，它更强调压缩轨迹对原始运动方向变化的保持能力。该方法将轨迹简化定义为 Min-Error 问题，即在给定存储预算W的条件下，选择不超过W个点构成压缩轨迹，并使压缩前后的最大方向误差最小。具体而言，Error-Search 首先构造所有可能线段产生的方向误差集合，并利用误差约束的单调性进行二分搜索；对于每个候选误差值，算法会判断是否存在一条满足该约束且保留点数不超过W的简化轨迹，从而逐步逼近最小可行误差。

该方法的优势在于能在存储预算固定的条件下，从全局角度优化轨迹的方向保持效果。它不需要用户预先指定误差阈值，而是根据可保留的点数自动搜索误差较小的结果。例如，在出租车轨迹分析中，若系统只能保留固定数量的点，Error-Search 能优先选择最大程度反映车辆行驶方向变化的关键点，避免转弯、绕行等重要运动趋势被过度压缩。然而，由于需要构造较大的误差搜索空间并多次执行可行性检查，其计算和存储开销相对较高，因此同样更适合离线或中小规模的轨迹压缩场景。

3.4.3. 基于深度学习的轨迹压缩

传统轨迹压缩方法通常围绕人工设计的距离误差、点重要性或搜索规则展开，例如递归分裂类方法依据几何偏差选择关键点，逐步合并类方法通过局部删除或片段合并减少冗余点，优化搜索类方法则从全局角度寻找误差较小的简化结果。这些方法具有较好的可解释性和实用性，但其压缩依据多由预设规则决定，主要关注轨迹几何形态的保持，对轨迹中的时间依赖、运动模式、区域语义以及下游任务需求刻画相对有限。

随着轨迹数据规模扩大和应用场景复杂化，轨迹压缩不再只是简单减少采样点，还需要尽可能保留对分析、预测和决策有价值的潜在信息。因此，深度学习方法被引入轨迹压缩研究，通过神经网络从数据中自动学习轨迹的空间结构、时序变化和行为特征，使压缩过程由规则驱动逐渐转向数据驱动。根据模型思想和压缩目标的不同，相关方法可分为自编码器与表示学习类、强化学习类以及任务驱动型轨迹压缩方法。

自编码器与表示学习类方法是深度学习轨迹压缩中的重要研究思路。与传统方法直接依据几何误差、距离阈值或点重要性删除冗余采样点不同，这类方法通常将轨迹看作由空间位置、时间戳或网格单元构成的序列，并利用神经网络学习其低维表示。其核心思想是通过编码器将原始序列映射为紧凑的向量表示，再借助解码器、相似性约束或下游任务目标，尽可能保留原始轨迹中的空间结构、时间依赖与运动模式。由此，轨迹压缩不再仅表现为采样点数量的减少，更表现为将复杂、变长的轨迹序列转化为固定维度的语义表示。

通常情况下，该类方法首先需对原始轨迹进行表示转换，例如将连续 GPS 点映射为离散网格编号，或将其空间位置、时间信息及运动属性转化为嵌入向量。随后，模型利用循环神经网络[33]、序列到序列（Sequence-to-Sequence，Seq2Seq）模型[34]、自编码器[35]或 Transformer[36]等结构对序列进行编码，获取固定或较低维度的轨迹表示。该表示既可作为压缩后的表达形式，以减少原始序列带来的存储与计算开销，也能进一步服务于下游任务。

CVAE[37]是这类轨迹压缩的代表性方法之一。其将原始车辆轨迹输入编码器，由编码器

提取轨迹中的时序变化特征和空间运动特征；随后将这些特征压缩为低维上下文向量；最后由解码器根据该上下文向量恢复车辆轨迹，从而实现轨迹数据的压缩表达。

如图 3.16 所示，CVAE 主要由轨迹输入、编码器、上下文向量和解码器四个模块组成。轨迹输入模块负责将车辆运动过程表示为连续的二维坐标序列，其中每个时间步包含车辆的横向和纵向位置信息，是后续特征学习的基础。编码器模块负责对原始轨迹序列进行特征提取和降维，将较长的轨迹序列逐步映射到低维潜在空间中。上下文向量模块位于编码器与解码器之间，是整个模型的压缩瓶颈，用于以较少的维度保存轨迹重构所需的关键信息。由于该方法采用变分自编码器结构，上下文向量并非简单的确定性特征，而是由潜在空间中的概率分布采样得到，因此能够更平滑地刻画不同轨迹之间的连续变化关系。

解码器模块则根据上下文向量恢复原始轨迹序列。它接收压缩后的低维表示，并逐步生成与输入轨迹长度一致的二维坐标序列，使压缩后的表示能够重新还原为可理解的车辆运动轨迹。为了保证恢复轨迹既接近原始轨迹，又符合车辆运动的连续性特征，模型在训练过程中通常同时约束轨迹重构误差、轨迹变化趋势误差以及潜在空间分布，使解码结果能够尽可能保留原始轨迹的主要运动形态。与传统轨迹压缩方法直接删除采样点不同，该方法并不显式判断某个点是否保留，而是通过编码器和解码器自动学习整条轨迹的潜在运动模式，并用低维上下文向量完成轨迹压缩。

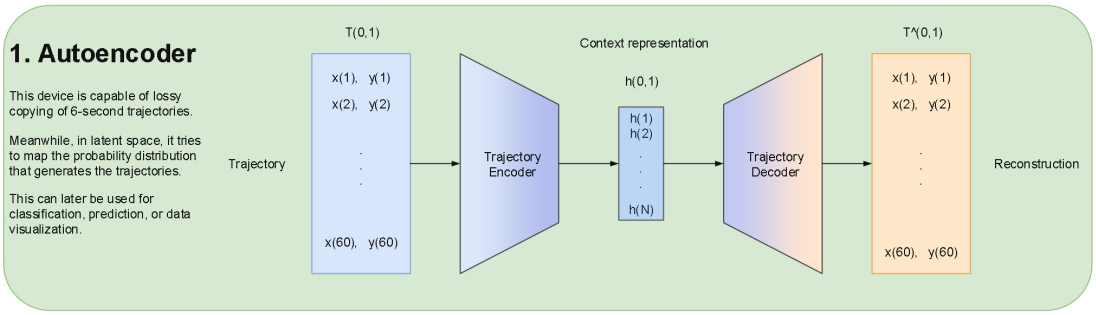
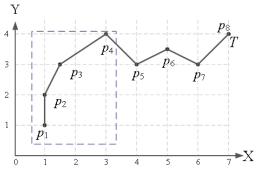


图 3.16 CVAE 的框架图

强化学习类轨迹压缩方法将轨迹压缩过程看作一个序列决策问题，其核心思想是通过学习策略自动判断哪些轨迹点应当保留，哪些轨迹点可以删除。与传统压缩方法依赖人工设计的误差阈值、重要性函数或启发式规则不同，强化学习方法通常通过状态、动作和奖励函数来刻画轨迹压缩过程，使模型能够在与轨迹数据的交互中学习点删除策略，并在压缩率和信息损失之间进行权衡。具体而言，模型在轨迹点逐步到达或完整轨迹已知的条件下，根据当前轨迹状态选择删除、保留或跳过某些轨迹点，并根据该操作对压缩误差和压缩效率的影响获得奖励，从而不断优化压缩策略。

在此类方法中，RLTS[38]是具有代表性的轨迹压缩方法。该方法将轨迹简化问题建模为马尔可夫决策过程[39]，并通过策略学习实现数据驱动的轨迹点删除。如图 3.17 所示，在在线压缩模式下，轨迹点按照时间顺序不断到达，并被依次存入固定大小的缓冲区 W。当缓冲区达到容量限制时，模型需要从候选轨迹点中选择一个点进行删除，以便为后续新到达的轨迹点留出空间。例如，图中缓冲区包含轨迹点 $\langle p_1, p_2, p_3, p_4 \rangle$，当缓冲区已满时，算法需要判断其中哪个点对整体轨迹表示的贡献较小，并将其作为删除对象。



Initial	Store $< p_1, p_2, p_3, p_4 >$ into the buffer with the initial reward 0.0				
Point	State	Action	Buffer	Reward	Accum Reward
p_5	$s_1 = \{v(p_2) = 0.243, v(p_3) = 0.354, v(p_4) = 1.0\}$	Drop p_3	$< p_1, p_2, p_4, p_5 >$	-0.354	-0.354
p_6	$s_2 = \{v(p_2) = 0.693, v(p_5) = 0.728, v(p_4) = 1.265\}$	Skip p_6 and p_7	$< p_1, p_2, p_4, p_5 >$	-0.278	-0.632
p_7	-			-	-
p_8	$s_3 = \{v(p_2) = 0.693, v(p_5) = 1.0, v(p_4) = 1.265\}$	Drop p_2	$< p_1, p_4, p_5, p_8 >$	-0.061	-0.693
Output	Return $T' = < p_1, p_4, p_5, p_8 >$ with $\varepsilon(T') = 0.693$				

图 3.17 RLTS 轨迹压缩过程示意图

具体而言，RLTS 首先为缓冲区中的轨迹点计算重要性值，该值用于衡量删除该点后可能引入的压缩误差。通常情况下，重要性较低的轨迹点说明其对轨迹形态、方向变化或时间同步信息的贡献较小，因而更适合作为候选删除点。随后，算法选取若干重要性较低的轨迹点构成当前状态，并将该状态输入策略网络，由策略网络输出删除不同候选点的概率。模型的奖励函数则根据删除操作对最终压缩误差的影响进行设计：若某次删除操作能够使压缩轨迹在较低点数下保持较小误差，则该动作获得较高奖励；反之，若删除操作导致轨迹形态或运动特征明显失真，则奖励较低。通过策略梯度等强化学习方法不断训练后，RLTS 能够学习到一种数据驱动的点删除策略，并应用于在线或批处理轨迹简化任务。

在基本 RLTS 的基础上，RLTS-Skip 进一步在动作空间中增加了“跳过若干点”的操作。当模型判断某些连续到达的轨迹点信息量较低、对轨迹整体形态贡献有限时，可以直接跳过这些点，而不必将其逐一加入缓冲区后再进行删除。例如，当车辆处于长时间直线或近似匀速行驶状态时，连续采样点之间的变化较小，模型可以通过跳过操作减少不必要的点级决策。通过这种方式，RLTS-Skip 既减少了在线压缩过程中的决策次数，也提高了轨迹流处理效率。

此外，RLTS 方法还可以扩展到批处理压缩场景。相关研究在基本 RLTS 的基础上提出了 RLTS+、RLTS++及其 Skip 版本，通过利用更多已扫描轨迹点或完整轨迹信息来构建状态，使模型能够在更充分的上下文条件下进行点删除决策。与在线模式相比，批处理模式能够利用更多未来轨迹信息，因而通常可以获得更好的压缩质量；而在线模式则更适合实时轨迹传输、移动设备定位和车辆运行监控等对响应速度要求较高的场景。不同版本在可用上下文信息、压缩质量和运行效率之间形成了不同折中。

任务驱动型轨迹压缩是深度学习中更直接面向压缩结果和应用需求的一类方法。与上述方法不同，任务驱动型方法通常直接面向关键点选择、简化结果或下游分析效果进行优化。其核心思想是利用神经网络自动学习轨迹点的重要性，使模型在减少数据量的同时，尽可能保留原始轨迹的空间结构、时间变化和语义上下文信息。换言之，这类方法不仅关注“如何表示轨迹”，更关注“哪些点应该被保留，以及压缩后的轨迹能否支撑后续分析任务”。

S3[40]是一种自监督轨迹简化方法。针对传统算法依赖人工规则、误差计算开销大且难以捕捉全局移动模式等问题，该方法提出了一个轻量级的序列到序列再到序列（Seq2Seq2Seq）框架。如图 3.18 所示，该框架由两个相互连接的 Seq2Seq 模型（Compressor 和 Constructor）组成。其中，Compressor 接收原始轨迹$\hat{T}$，并从中选择若干关键点生成简化轨迹T'；Constructor 则以 T'为输入，尝试重构原始轨迹$\hat{T}$。模型通过比较$\hat{T}$与$\hat{T}$之间的差异构造重构损失$L_R(T,\hat{T})$，并以此进行自监督训练。通过这种方式，Compressor 能在无人工标注的情况下，自动学习哪些点对恢复原始轨迹更为重要，从而实现压缩。

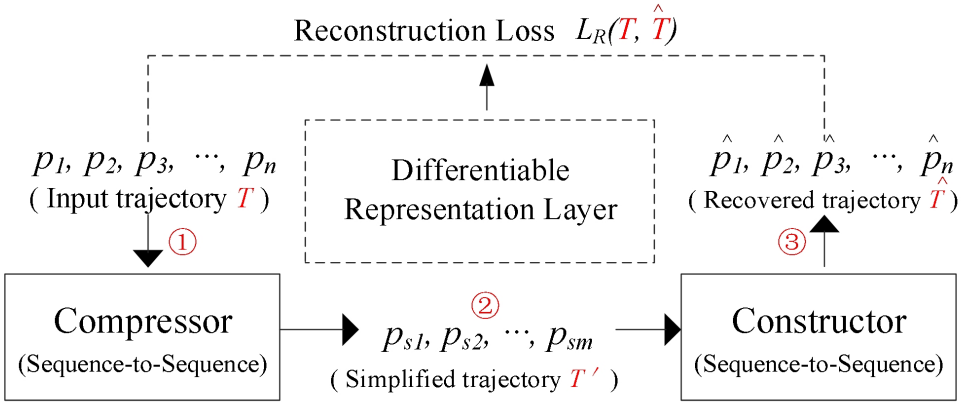


图 3.18 S3 的架构图

在具体结构上，S3 首先通过嵌入层将连续 GPS 点映射为离散网格，并利用 Skip-gram 方法学习网格单元的向量表示，使空间位置能以可学习的向量形式输入模型。随后，Compressor 采用带注意力机制的编码器—解码器结构，根据轨迹序列表示生成指定长度的简化轨迹；Constructor 再据此恢复原始序列。由于关键点选择本质上属于离散操作，直接进行端到端训练较为困难，因此 S3 引入了可微分层，通过 Gumbel–Softmax 等方式连接 Compressor 和 Constructor。具体而言，Compressor 负责从原始轨迹中选择关键点，Constructor 则负责检验这些点是否足以恢复原始信息。若压缩后的轨迹能较好地重构原始轨迹，说明保留的点包含了较多关键运动信息；反之，则说明丢失了重要的空间或时间特征。借助这一自监督机制，S3 能够在无压缩标签的条件下学习轨迹点的重要性。

此外，S3 还通过上下文感知图神经网络增强了轨迹表示能力。具体而言，该方法将空间区域划分为网格，根据轨迹在相邻网格间的转移关系构建上下文空间图，并利用图卷积网络学习网格间的全局移动依赖关系。这样，模型不仅能关注单个点的位置特征，还能捕捉其所在区域间的上下文移动模式。

S3 的优势在于能以自监督方式进行轨迹简化，既无需预先构造人工标签，也不完全依赖人工设计的误差规则。但该方法也存在一定的局限性：一方面，其压缩目标主要由重构损失等指标引导，因此在 PED、SED 和方向角距离（Direction-aware Distance，DAD）等误差指标上未必始终最优；另一方面，模型仍需针对具体数据集进行训练，实际应用效果会受到网格划分粒度、训练数据质量、采样特征以及上下文图构建方式等因素的综合影响。

3.5 轨迹恢复

在轨迹数据管理与分析流程中，轨迹恢复是针对轨迹缺失、采样稀疏和观测不连续等问题的重要处理环节。其核心任务是根据已有采样点、历史运动规律、道路网络约束或上下文环境信息，对缺失的位置、片段或可能经过的路径进行补全与重建，从而提升数据的完整性。由于轨迹具有显著的时空连续特征，采样点缺失或间隔过大不仅会造成局部运动信息丢失，还可能破坏整体的路径连贯性，进而影响地图匹配、路径重构、交通状态估计、行为识别和出行模式挖掘等后续任务。因此，在轨迹治理流程中，有必要根据数据质量状态和应用目标，对不完整或低采样率轨迹进行合理的恢复处理。轨迹恢复可位于清洗之后，也可在压缩、建模或地图匹配之前完成。

与简单的数据补全或几何插值不同，轨迹恢复并非单纯根据相邻观测点的位置关系进行线性填补，而是需综合考虑时间顺序、空间可达性、道路拓扑结构、移动对象运动规律及历史模式等多方面因素。其目标并非生成唯一确定的真实路径，而是在观测信息有限的条件下，推断出最符合实际运动过程的可能轨迹。恢复方法和模型参数应根据数据来源、采样频率、移动对象类型、路网质量及具体应用目标进行合理设定。合理设置方法与参数，有助于在补全缺失信息的同时，尽可能保持轨迹的时空合理性和路径合理性，为后续的分析、交通建模和行为理解提供可靠的数据基础。

3.5.1. 轨迹恢复基本概述

轨迹恢复是指针对原始数据中存在的点缺失、片段缺失、采样间隔过大或观测不连续等问题，依据相邻观测点、运动规律、道路网络、历史模式或上下文信息，对缺失位置或路径

片段进行补全与重建的处理方法。其核心目标在于提升数据的完整性、连续性与可用性，使恢复后的轨迹能更准确地反映移动对象在时空维度上的真实运动过程。轨迹数据通常由带有时间戳和空间坐标的采样点按时间顺序排列而成，各点之间具有连续的时空关联。因此，轨迹恢复不仅要关注缺失位置的补全，还需判断恢复结果是否符合实际运动规律、道路可达性与路径连续性要求。

从处理流程来看，轨迹恢复通常包括缺失情况识别、已有信息分析、候选路径或位置生成、缺失推断及结果评价等环节。首先，需根据采样点的时间间隔、空间距离和采样频率，判断数据中是否存在缺失点、缺失片段或低采样问题。其次，应结合相邻观测点的位置关系、移动对象的速度与方向变化、历史模式及道路拓扑结构，推断缺失时段内可能经过的位置序列或路径。此外，对于生成的恢复结果，还需通过空间偏差、时间一致性、路径可达性、速度合理性和道路匹配程度等指标评估其质量，以此判断是否满足研究与应用需求。

相较于一般的数据补全，轨迹恢复的特殊性主要体现在保持时空连续性与推断路径合理性两个方面。一方面，恢复后的轨迹应尽可能保持原始运动的时间连续性，即缺失点或片段的补全结果应与前后观测点在时间顺序、速度变化和运动趋势上保持一致。另一方面，恢复后的轨迹还应满足空间可达性与道路约束，即推断出的位置或路径应符合移动对象的实际运动能力与路网通行条件。例如，在车辆轨迹中，若仅依据相邻观测点进行直线插值，可能会生成穿越建筑物、河流等不可通行区域的轨迹；而在低采样率的出租车轨迹中，若忽略道路连通关系与行驶速度约束，则可能恢复出与真实路线明显不符的路径。

此外，轨迹恢复并非孤立的数据处理步骤，而是预处理与数据完整性增强中的重要环节。其结果将直接影响地图匹配、路径重构、交通状态估计、行为识别、出行模式挖掘和轨迹预测等下游任务。如果恢复不足，缺失信息将继续传递至后续分析中，导致路径不完整、运动特征丢失及行为模式判断偏差；如果恢复过度或不合理，则可能引入虚假的采样点或错误路径，反而降低数据的真实性与分析结果的可靠性。因此，在实际应用中，应结合数据来源、采样频率、移动对象类型、路网质量及具体应用场景合理制定恢复策略，确保在补全缺失信息的同时，尽可能保持轨迹的时空连续性、路径合理性与真实运动特征。

3.5.2. 传统轨迹恢复

早期轨迹恢复研究主要集中于基于几何与概率的恢复方法，包括传统几何方法和基于路网与概率图的方法。前者通常依据轨迹点之间的空间位置关系对缺失轨迹进行插值与重建，后者则结合道路网络结构与概率模型，对车辆可能的行驶路径进行推断与恢复。

该类方法主要依据相邻观测点之间的空间位置和时间关系对缺失轨迹进行补全。线性插值法假设移动对象在相邻观测点之间以匀速直线运动，并按照时间比例在两点之间插入新的轨迹点；样条插值等方法则通过构造平滑曲线恢复连续运动过程。该类方法实现简单、计算效率较高，适合采样间隔较短且运动变化较平滑的场景。但由于其主要依据几何位置关系进行恢复，通常难以考虑道路连通性、交通规则和实际可达路径，因此在车辆轨迹恢复中可能产生偏离道路或穿越不可达区域的结果。

该类方法将道路网络抽象为由节点和边组成的图结构，并利用道路连通关系恢复相邻观测点之间的可能行驶路径。常见做法是先将观测点匹配到候选道路段或路网节点，再利用 Dijkstra 算法[41]、A*搜索算法[42]等最短路径算法在相邻候选位置之间搜索可达路径，从而推断车辆可能经过的道路序列。相比单纯几何插值，基于路网约束的方法能够更好地保证恢复轨迹符合道路拓扑结构和行驶可达性，是车辆轨迹恢复中的基础方法。但若边权仅使用道路长度，这类方法偏向较短路线；使用行驶时间等权重则对应不同目标，难以充分刻画观测误差、行驶速度、方向变化和路径选择不确定性。

基于路网与概率图模型的轨迹恢复是传统方法中较为重要的一类。与几何插值主要依据相邻点位置进行补全、单纯路网约束方法主要依赖最短路径搜索不同，该类方法通常将观测点、候选路段及道路间的转移关系统一纳入概率推断框架，从而对稀疏或缺失轨迹进行全局恢复。其基本思想是：首先为每个 GPS 观测点生成若干候选路段或候选状态；然后结合道路拓扑结构、观测点到候选路段的空间距离、相邻点间的时间间隔、速度约束与路径可达性等因素，构建候选状态图或概率图模型；最后从中选取整体概率最大或综合得分最高的路径序列作为恢复结果，典型算法包括基于隐马尔可夫模型（Hidden Markov Model，HMM）的地图匹配方法[2]和 ST-Matching[43]等。

基于 HMM 的地图匹配方法是该轨迹恢复的典型代表。这类方法通常将车辆真实行驶的路段视为隐状态，将 GPS 观测点视为观测序列，并通过概率推断寻找最可能的状态序列。具体而言，首先在每个观测点附近生成候选路段，并根据观测点到候选路段的距离计算发射概率。通常，观测点距离某候选路段越近，说明该观测点由该路段产生的可能性越大，对应的发射概率也越高。随后，模型根据相邻候选路段之间的连通关系、路网距离、直线距离以及时间间隔等因素计算转移概率。一般认为，实际行驶的路网距离与相邻观测点间的直线距离越接近，状态转移越合理。最后，模型利用维特比算法在候选状态序列中寻找整体概率最大的路段序列，得到模型假设下最可能的道路序列。

如图 3.19 所示，这类方法首先为每个观测点生成候选状态，计算发射概率及相邻状态间的转移概率，进而在候选状态序列中搜索最优路径。其优势在于能够以统一的概率框架同时处理测量噪声与道路连通约束，避免局部匹配错误。然而，该方法的恢复效果高度依赖候选路段生成策略和概率建模方式：当采样间隔过大、候选路径过多或路网结构较复杂时，匹配的不确定性与计算开销都会显著增加。

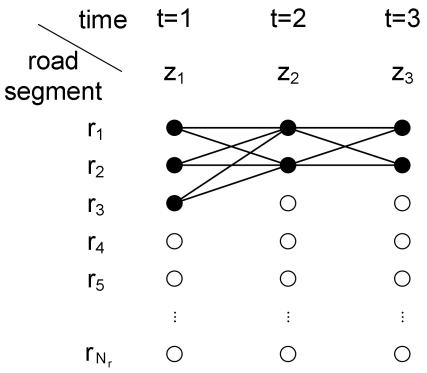


图 3.19 HMM 轨迹恢复中的候选道路状态序列

ST-Matching 则是一种面向低采样率 GPS 轨迹的全局地图匹配与恢复方法。与 HMM 主要通过概率刻画观测点与候选路段的匹配关系不同，它更强调空间与时间因素的联合分析。该方法不仅考虑点到候选路段的空间距离，还综合评估候选路径的拓扑连通性、绕行程度、相邻点间的时间间隔，以及由此推导出的速度合理性。通过同时引入时空约束，ST-Matching 能在低采样率下推断出更符合实际运动的路径。

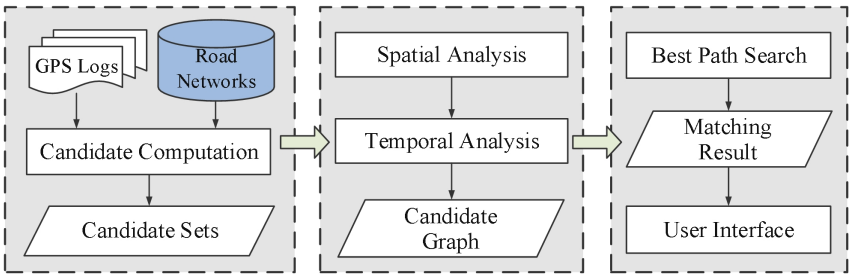


图 3.20 ST-Matching 轨迹恢复方法流程

图 3.20 展示了 ST-Matching 的恢复流程。首先在每个观测点附近搜索候选路段，并将观测点投影至其上形成候选点；然后，计算相邻观测点的候选点之间的候选路径，并分别进行空间与时间分析。空间分析主要衡量观测点与候选道路的距离关系及路径是否存在明显绕行；时间分析则利用时间间隔与路径长度，判断对应的行驶速度是否合理。最后，构建候选图，并在图中搜索整体得分最高的路径序列作为恢复结果。

该方法的优势在于能较好地适应低采样率场景。例如，出租车可能每隔数分钟才记录一次位置，此时相邻点间往往存在多条可行路径，单纯依靠最近道路或局部匹配极易出错；而

ST-Matching 结合了可达路径、空间匹配度及速度约束，能恢复出更合理的路线。但它也存在一定局限：由于需进行候选路径搜索与构建候选图，计算成本受候选点数量、路网规模及最短路径计算复杂度的影响较大；同时，恢复效果也高度依赖搜索半径、速度约束的设置以及路网数据本身的质量。

3.5.3. 基于深度学习的轨迹恢复方法

相较于基于几何与概率的轨迹恢复方法，基于深度学习的轨迹恢复方法在训练数据充足且路网信息可用的条件下，通常具有更强的时空模式学习能力和更细粒度的恢复潜力，主要包含基于 LSTM、GRU 等循环神经网络结构的序列建模方法[44]、基于 GAN 的插补模型[45,46]以及基于 Transformer 结构的轨迹恢复模型[48]，以及面向特定任务设计的恢复框架。

序列生成类方法是深度学习轨迹恢复中较为典型的方法。该类方法通常将恢复任务看作从低采样轨迹到高采样轨迹的序列映射过程，即根据稀疏的观测点，生成目标采样率下更完整、连续的点序列。其基本思想是：利用编码器对输入的低采样轨迹进行建模，提取已有观测点的空间位置、时间顺序和运动上下文信息，形成轨迹级隐表示；随后，解码器以该表示为基础，按时间顺序逐步预测缺失点或高采样率序列。与传统插值相比，序列生成类方法不再简单假设移动对象在相邻点之间保持匀速直线运动，而是通过训练数据学习时空依赖关系和运动模式，因而更适合处理采样间隔不均匀、运动状态复杂或缺失片段较长的数据。

在此类研究中，MTrajRec[44]是一种代表性的序列生成模型。针对传统“两阶段”恢复流程易累积误差且效率较低的问题，该方法提出了一种端到端的地图约束轨迹恢复框架。传统流程通常先在自由空间中恢复高采样率点，再利用地图匹配算法将其投影至路网上。这种做法虽然直观，但前一阶段产生的位置误差极易在匹配阶段被进一步放大，导致最终路径偏离真实道路。相比之下，MTrajRec 将缺失点恢复与地图匹配统一在同一模型内，使其在生成关键点的同时直接考虑道路约束，减少了两阶段处理中的误差传递。

MTrajRec 将低采样轨迹恢复类比为机器翻译任务。在这一视角下，低采样轨迹可视为源语言句子，目标采样率下的地图匹配轨迹则视为目标语言句子，恢复过程即对应源序列到目标序列的翻译。基于此，该方法采用 Seq2Seq 框架对低采样数据进行编码，以生成高采样率的道路约束轨迹。图 3.21 展示了 MTrajRec 的模型架构。该模型采用多任务 Seq2Seq 结构，由编码器和解码器两部分组成。编码器首先将低采样 GPS 点转换为离散网格序列，并加入时间索引信息，使模型能感知相邻点间的时间间隔，从而判断需恢复的缺失点数量；随后，利用 GRU 对输入序列进行编码，提取其中的时空依赖关系，形成上下文向量。

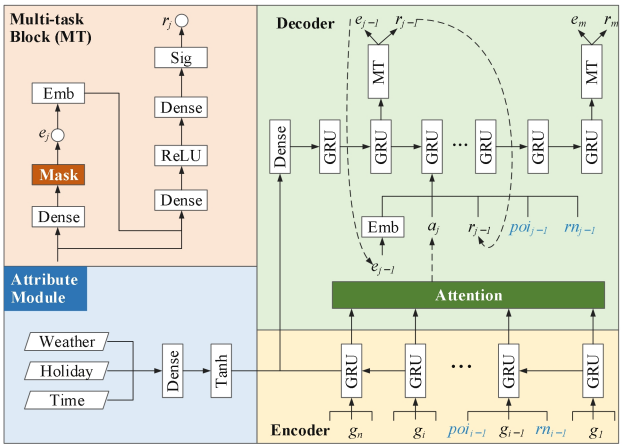


图 3.21 MTrajRec 轨迹恢复模型结构

在解码阶段，MTrajRec 以该上下文向量为初始状态，按目标采样率逐步生成恢复结果。与直接预测经纬度坐标不同，它将每个恢复点表示为$\langle e, r, t\rangle$，其中e表示路段编号，r表示该点在对应路段上的移动比例，t表示时间戳。通过同时预测路段编号及其内部的位置比例，模型能使恢复结果自然约束在道路网络上，避免生成偏离道路的自由空间坐标。由此，MTrajRec 在一个统一框架内同步完成了点恢复与地图匹配，实现了从低采样轨迹到高采样地图匹配轨迹的端到端转换。

为进一步提高恢复精度，MTrajRec 在基础 Seq2Seq 结构上引入了多个增强模块。首先是约束掩码层，它根据原始 GPS 点到候选路段的距离来限制可选路段范围，避免模型在整个路网空间中进行无约束预测，从而缓解了离散网格表示可能带来的空间精度损失。其次是注意力机制，用于建模解码过程中当前恢复点与各输入观测点之间的全局关联，使模型在生成缺失点时能更充分地利用上下文信息。最后是属性模块，进一步引入了天气、节假日、时间、兴趣点分布及道路属性等外部因素，以刻画交通状态和环境条件对车辆运动的影响。通过这些模块，MTrajRec 能在路网约束下恢复出更细粒度的轨迹点，有助于提升恢复结果的空间合理性与时序连续性。

相比之下，对抗生成类恢复方法则主要借鉴生成对抗网络的思想，将轨迹恢复视为生成模型与判别模型相互博弈的过程。一般而言，生成器根据稀疏观测数据、道路环境或运动状态，生成缺失点、片段或完整序列；判别器则用于评估生成结果与真实轨迹的差异，并提供反馈信号。通过持续的对抗训练，生成器逐渐学习真实数据的分布特征，使恢复轨迹在空间位置、时间连续性与运动行为上更贴近真实情况。与几何插值或普通监督预测相比，该类方法不仅关注单个缺失点的预测误差，更强调恢复结果在整体分布与行为模式上的真实性，因而更适合处理观测稀疏、路径选择不确定性强或运动行为复杂的恢复任务。

在此类研究中，ImIn-GAIL[47]是一种代表性的对抗式模仿学习框架。如图 3.22 所示，该方法将轨迹插值与模仿学习整合于同一框架内，将插值作为行为学习中的辅助环节。具体而言，ImIn-GAIL 主要由生成器、下采样器和插值—判别器三部分组成。首先，生成器在仿真环境中根据当前驾驶状态生成车辆动作，形成高采样率轨迹；其次，下采样器从中抽取稀疏数据，以模拟真实场景的稀疏观测过程，并构造出从稀疏到稠密的训练样本；最后，插值—判别器同步承担恢复与判别双重任务，一方面根据稀疏专家轨迹恢复未观测的稠密点，另一方面判断插值后的专家轨迹与生成轨迹是否源于同一真实行为分布。

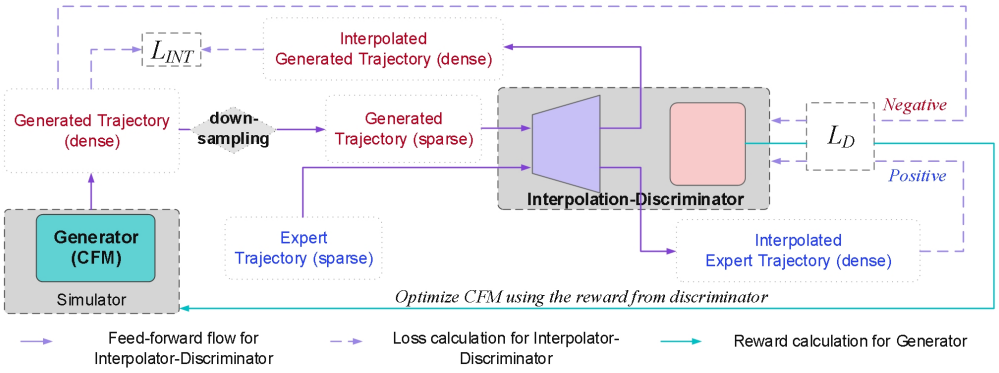


图 3.22 ImIn-GAIL 轨迹恢复框架

通过这种联合训练方式，ImIn-GAIL 实现了插值模块与模仿学习模块的相互促进。一方面，模型使用生成器产生的高采样序列，学习从稀疏到稠密的恢复映射关系；另一方面，更合理的插值结果能为判别器提供更完整的专家行为信息，进而提升对生成策略的监督效果。相比于将插值作为独立预处理步骤的传统做法，ImIn-GAIL 有效阻断了误差向后续学习过程的传播，使模型在稀疏观测条件下能更精准地恢复车辆在未观测时段内的运动轨迹。

Transformer 类轨迹恢复方法将低采样恢复视为从稀疏观测序列到高采样序列的建模过程，其核心思想是利用自注意力机制捕捉点之间的长程依赖关系。与 LSTM、GRU 等循环神经网络按时间顺序递归建模不同，Transformer 能同时关注序列中不同位置的观测点，从而更充分地利用全局上下文信息。在低采样场景中，相邻观测点间往往存在较长时间间隔，缺失点较多，仅依赖局部时序关系难以准确推断真实的运动过程。因此，Transformer 可用于建模长距离的时空关联，并结合时间间隔、路网拓扑与上下文信息来精准恢复缺失点。

RNTrajRec[48]是一种典型的 Transformer 类轨迹恢复方法。该方法认为，低采样轨迹的恢复不仅需要建模点之间的时间依赖关系，还需显式引入路网结构约束，以保证结果符合实际道路拓扑与空间可达性。为此，RNTrajRec 提出了一个结合路网表示学习与时空 Transformer 的端到端框架。该框架一方面利用图神经网络学习路段间的空间邻近与拓扑连接关系，另一

方面利用 Transformer 建模低采样序列中的全局时序依赖，从而实现时间动态与空间约束的联合建模。

在路网表示阶段，RNTrajRec 首先利用 GridGNN 学习路段表示。具体而言，模型将每条路段表示为其经过的网格序列，并结合图注意力网络捕捉路段间的拓扑关系，从而获得包含空间邻近性与连通性的道路表示。随后，针对输入序列中的每个 GPS 观测点，模型通过子图生成模块提取其周围的相关路段，构造局部道路子图，以描述该点附近的道路环境。通过这种方式，模型不仅能感知观测点本身的位置特征，还能获取其周围的道路结构与候选路径信息，为后续恢复提供坚实的空间约束。

图 3.23 展示了 RNTrajRec 的模型架构。在编码阶段，RNTrajRec 设计了核心模块 GPSFormer。它由多个 GPSFormerBlock 堆叠而成，每个 Block 包含 Transformer 编码层和图细化层两部分。其中，前者用于建模序列中不同观测点间的全局时间依赖关系，使模型能捕捉相距较远点间的运动关联；后者则结合每个 GPS 点对应的局部道路子图，对空间表示进行进一步细化。GPSFormer 一方面通过自注意力机制学习点间的长程时序关系，另一方面通过图结构引入周围的路网拓扑信息，从而同步刻画轨迹的时间动态与空间约束。

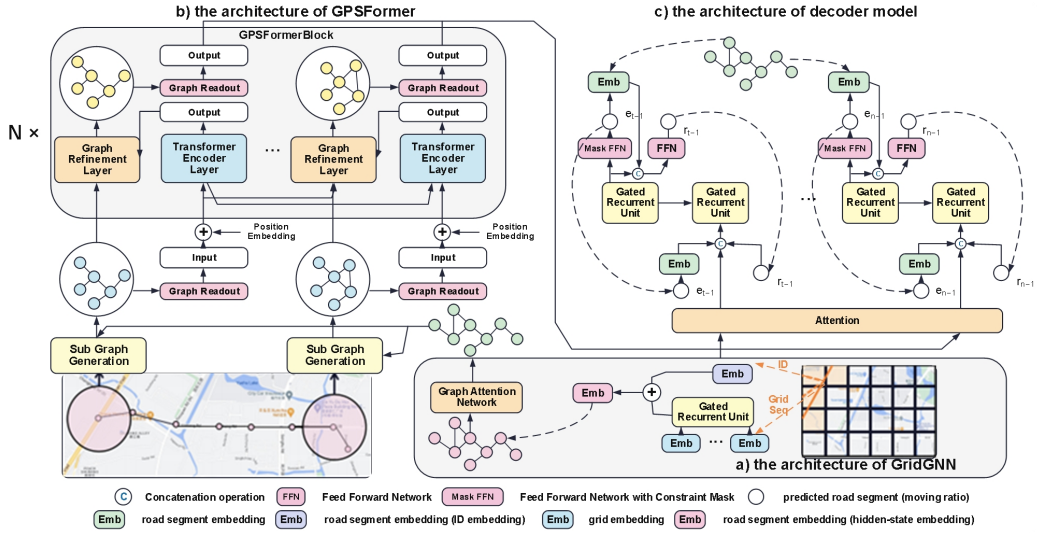


图 3.23 RNTrajRec 轨迹恢复模型结构

在解码阶段，RNTrajRec 采用多任务解码器恢复缺失点。模型首先根据 GPSFormer 输出的表示，通过注意力机制获得解码所需的上下文向量；随后利用 GRU 按目标采样率逐步生成高采样率轨迹点。与直接预测经纬度坐标不同，RNTrajRec 同时预测目标点所在的路段及其在该路段上的移动比例，从而使预测点落在所选路段上，但所选路段仍可能错误，点位在路网上也不保证相邻路段连通。此外，模型还引入了约束掩码与图分类损失，引导其在候选道路范围内进行更合理的预测，进一步提高了恢复结果的空间一致性与地图匹配准确性。

任务驱动型轨迹恢复方法通常围绕特定数据来源、应用场景或业务需求进行模型设计。与通用轨迹恢复方法主要处理低采样 GPS 轨迹不同，这类方法面对的数据形式和恢复目标往往更加复杂。例如，在城市交通场景中，轨迹数据可能并不直接来自连续 GPS 记录，而是来自交通摄像头采集的车辆快照、过车记录、车牌识别结果或其他城市感知数据。由于这类数据通常存在观测稀疏、设备覆盖不足、识别结果不确定、车辆身份关联困难以及路网信息不完整等问题，因此轨迹恢复过程不仅需要补全缺失路径，还需要结合具体场景对数据预处理、身份关联、转移概率建模和路径生成等环节进行联合设计。相较于通用方法，任务驱动型轨迹恢复方法更强调城市交通应用中的可用性和适应性，可服务于私家车轨迹恢复、交通流监测、道路运行分析和信号控制优化等场景。

TrajRecovery[49]是一种典型的任务驱动型轨迹恢复框架。该方法面向城市级交通摄像头过车记录场景，不依赖连续车辆 GPS 轨迹，而是利用交通摄像头持续采集的车辆快照和过车记录恢复完整车辆轨迹。与基于 GPS 点序列的轨迹恢复不同，TrajRecovery 需要处理摄像头覆盖不均、过车记录稀疏、车辆身份识别误差以及不同驾驶者路径选择偏好差异等问题。为此，该方法将轨迹恢复流程划分为三个主要阶段：预处理阶段、空间转移概率建模阶段和轨迹生成阶段。图 3.24 展示了 TrajRecovery 包含数据预处理、核心概率建模和轨迹路径生成在内的整体系统框架。

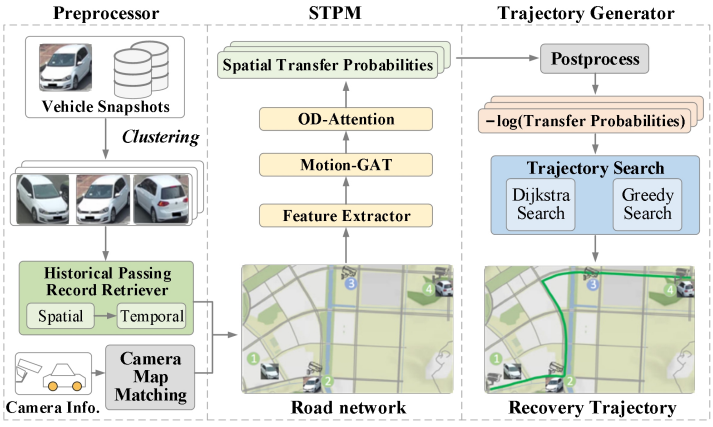


图 3.24 TrajRecovery 轨迹恢复系统框架

在预处理阶段，TrajRecovery 首先需要将交通摄像头与道路网络中的具体道路段建立对应关系。由于摄像头的经纬度、名称和拍摄方向可能存在缺失或错误，系统会根据摄像头位置信息的可信程度采用不同的匹配策略，将摄像头尽可能准确地绑定到相应道路段上。随后，模型利用车辆快照中的外观特征和车牌特征进行车辆聚类，从而将分散在不同摄像头处的过车记录关联为同一车辆的观测序列。考虑到摄像头故障、遮挡或覆盖不足可能导致某些过车

记录缺失，TrajRecovery 进一步设计了历史过车记录检索模块，通过查找与当前车辆行驶模式相似的历史记录，为后续轨迹恢复提供更充分的观测依据。

在核心建模阶段，TrajRecovery 提出空间转移概率模型，用于估计车辆从当前道路段转移到相邻道路段的可能性。该模型首先通过特征提取器融合道路长度、道路宽度、道路等级、道路类型和时间信息等特征，得到道路段的时空表示。随后，模型引入 Motion-GAT 模块，将道路段之间的转向关系和邻接关系纳入建模，使模型能够学习不同道路条件对车辆转向选择的影响，并将摄像头覆盖区域中学习到的转移规律推广到未覆盖道路段。为了进一步刻画不同车辆之间的个体化出行差异，TrajRecovery 从车辆历史起讫点记录中提取路径选择偏好，使模型能够在相同起终点条件下恢复出更符合个体驾驶习惯的轨迹。

在轨迹生成阶段，TrajRecovery 将空间转移概率转化为道路网络上的路径搜索问题。具体而言，系统将道路段视为图中的节点，将道路段之间的连接关系视为边，并将道路转移概率的负对数作为边权重。这样，寻找最可能恢复轨迹的问题就可以转化为图上的最短路径搜索问题，进而利用 Dijkstra 算法搜索概率最大的完整路径。相比每一步选择局部最大转移概率道路段的贪心策略，该方法能够从全局角度生成更合理的恢复轨迹，减少局部最优和路径循环等问题，从而提升城市级车辆轨迹恢复的可靠性。

综合来看，传统轨迹恢复方法与深度学习轨迹恢复方法并不是简单的替代关系，而是适用于不同条件下的轨迹数据补全任务。传统方法具有较强的可解释性和较低的使用成本，适合作为基础方法或用于采样间隔较短、路网约束明确的场景；深度学习方法具有更强的表示能力和扩展潜力，适合处理低采样率、高噪声、复杂路网和多源感知数据下的轨迹恢复任务。未来的轨迹恢复研究将更加关注路网结构融合、个性化运动建模、可解释深度恢复、多源感知数据利用以及面向真实城市交通应用的高效恢复框架。

3.6 小结与展望

本章围绕轨迹大数据治理的核心问题，系统介绍了轨迹数据在质量评估、清洗、压缩与恢复等方面的基本概念、典型方法和技术发展趋势。轨迹数据不同于一般结构化数据，其本质上是一类具有时间顺序、空间连续性和运动规律约束的时空序列数据。因此，轨迹治理不仅需要处理缺失、重复、异常和格式不一致等常规质量问题，还需要综合考虑速度、加速度、方向变化、道路网络约束以及时空分布偏差等轨迹特有因素。通过质量评估，可以对轨迹数据的完整性、有效性、一致性、公平性和任务适配性进行诊断；通过轨迹清洗，可以识别并

修正噪声点、漂移点和不合理运动片段；通过轨迹压缩，可以在控制信息损失的前提下降低存储、传输和计算成本；通过轨迹恢复，则可以对缺失轨迹点或轨迹片段进行补全与重建，为后续轨迹挖掘和智能分析提供可靠数据基础。

展望未来，轨迹大数据治理将朝着更加智能化、实时化、语义化和任务驱动的方向发展。一方面，随着深度学习、图神经网络、Transformer 和生成式模型的发展，轨迹治理方法将具备更强的复杂时空模式学习能力，能够更好地适应低采样率、高噪声和多源异构轨迹数据场景。另一方面，单纯追求数据本身“干净”已难以满足实际应用需求，未来治理过程应更多结合交通预测、路径推荐、异常检测和城市治理等下游任务，形成面向任务效果的数据质量优化机制。同时，轨迹数据涉及个体出行行为和位置隐私，如何在提升数据可用性的同时保护用户隐私，也将成为轨迹大数据治理的重要研究方向。总体而言，轨迹大数据治理是连接原始数据采集与高层智能应用的关键环节，其方法体系的完善将持续推动智能交通、智慧城市和位置服务等领域的发展。

3.7 术语表

Trajectory Data	轨迹数据
Trajectory Point	轨迹点
Trajectory Segment	轨迹段
Spatio-temporal Continuity	时空连续性
Sampling Frequency	采样频率
Localization Drift	定位漂移
Trajectory Data Quality Assessment	轨迹数据质量评估
Task-agnostic Quality Assessment	任务无关质量评估
Task-aware Quality Assessment	任务感知质量评估
Temporal Sequence Anomaly	时间序列异常
Kinematic Logic Conflict	运动学逻辑冲突
Data Distribution Imbalance	数据分布不均衡
Trajectory Cleaning	轨迹清洗
Noise Point	噪声点
Irregular Movement Segment	不合理移动片段

Duplicate Record	重复记录
Temporal Order Correction	时间顺序校正
Hard Constraint	硬约束
Soft Constraint	软约束
Spatial Jump	空间跳跃
Anomaly Detection	异常检测
Smoothing	平滑处理
Map Matching	地图匹配
Trajectory Compression	轨迹压缩
Compression Ratio	压缩率
Information Loss	信息损失
Key Point	关键点
Redundant Sampling Point	冗余采样点
Trajectory Simplification	轨迹简化
Compression Error	压缩误差
Spatial Position Error	空间位置误差
Temporal Synchronization Error	时间同步误差
Direction Error	方向误差
Semantic Information Loss	语义信息损失
Synchronized Position	同步位置
Trajectory Recovery	轨迹恢复
Trajectory Imputation	轨迹插补
Missing Trajectory Segment	缺失轨迹片段
Sparse Trajectory	稀疏轨迹
Low-sampling-rate Trajectory	低采样率轨迹
Linear Interpolation	线性插值
Spatio-temporal Interpolation	时空插值
Path Recovery	路径恢复

Road Connectivity	道路连通性
Map-constrained Recovery	路网约束恢复
End-to-end Recovery	端到端恢复

3.8 习题

习题 3.1

轨迹大数据治理主要包括轨迹质量评估、轨迹清洗、轨迹压缩和轨迹恢复四类基础任务。请分别说明这四类任务解决的核心问题，并简述它们在轨迹数据处理流程中的先后关系。

习题 3.2

轨迹质量评估可分为“任务无关的数据质量评估”和“任务感知的数据质量评估”。请说明二者的根本区别，并结合轨迹预测或交通状态识别任务解释为何仅依赖任务无关评估是不够的。

习题 3.3

与 TsQuality、TFDV、F4U 等通用数据质量评估方法相比，T-Assess 如何考虑轨迹数据的特有约束？请说明其四个核心质量维度，并解释“公平性”维度在轨迹数据治理中的意义。

习题 3.4

轨迹清洗首先需要判断“什么是异常”。请从物理约束、统计离群和地图匹配三个角度说明轨迹异常的常见判定标准，并各举一个例子。

习题 3.5

SHoTClean 是结合软硬约束的多变量时间序列清洗方法。请说明其“硬约束”和“软约束”分别起什么作用，并解释其为何可被转化为有向无环图中的最大权重路径搜索问题。

习题 3.6

TranAD 利用 Transformer 结构进行多变量时间序列异常检测。请说明其两阶段重构机制，并讨论将其用于轨迹异常筛查时需要哪些特征、约束与验证步骤。

习题 3.7

Douglas-Peucker、SQUISH 和 Error-Search 分别代表三种不同思路的轨迹压缩方法。请比较它们的核心思想、适用场景与主要局限。

习题 3.8

RLTS 和 S3 均属于深度学习驱动的轨迹压缩方法。请说明它们各自将轨迹压缩建模为什么

问题，并比较二者在“点选择机制”上的不同。

习题 3.9

线性插值、ST-Matching 和 MTrajRec 分别代表几何恢复、基于路网与时空约束的恢复、以及深度学习恢复三种思路。请比较它们的核心假设、优势与局限。

参考文献

[1] Zheng Y, Liu L, Wang L, et al. Learning transportation mode from raw gps data for geographic applications on the web[C]//Proceedings of the 17th international conference on World Wide Web. 2008: 247-256.

[2] Newson P, Krumm J. Hidden Markov map matching through noise and sparseness[C]//Proceedings of the 17th ACM SIGSPATIAL international conference on advances in geographic information systems. 2009: 336-343.

[3] Wang R Y, Strong D M. Beyond accuracy: What data quality means to data consumers[J]. Journal of management information systems, 1996, 12(4): 5-33.

[4] Gao Cong, Wenfei Fan, Floris Geerts, Xibei Jia, and Shuai Ma. 2007. Improving Data Quality: Consistency and Accuracy. In VLDB. 315–326.

[5] Sebastian Schelter, Stefan Grafberger, Philipp Schmidt, Tammo Rukat, Mario Kießling, Andrey Taptunov, Felix Bießmann, and Dustin Lange. 2019. Differential Data Quality Verification on Partitioned Data. In ICDE. 1940–1945.

[6] Daniele Foroni, Matteo Lissandrini, and Yannis Velegrakis. 2021. Estimating the extent of the effects of Data Quality through Observations. In ICDE. 1913–1918.

[7] Qiu Y, Fang C, Song S, et al. TSQuality: measuring time series data quality in Apache IoTDB[J]. Proceedings of the VLDB Endowment, 2023, 16(12): 3982-3985.

[8] Caveness E, GC P S, Peng Z, et al. Tensorflow data validation: Data analysis and validation in continuous ml pipelines[C]//Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data. 2020: 2793-2796.

[9] Daniele Foroni, Matteo Lissandrini, and Yannis Velegrakis. 2021. The F4U System for Understanding the Effects of Data Quality. In ICDE. 2717–2720.

[10] Zhu J, Wang T, Hu D, et al. T-Assess: An Efficient Data Quality Assessment System Tailored for Trajectory Data[J]. Proceedings of the VLDB Endowment, 2024, 18(3): 666-674.

[11] Fei Gao, Shaoxu Song, and Jianmin Wang. 2021. Time Series Data Cleaning under Multi-Speed Constraints. Int. J. Softw. Informatics 11, 1 (2021), 29–54.

[12] Shaoxu Song, Fei Gao, Aoqian Zhang, Jianmin Wang, and Philip S. Yu. 2021. Stream Data Cleaning under Speed and Acceleration Constraints. ACM Trans. Database Syst. 46, 3 (2021), 10:1–10:44.

[13] Shaoxu Song, Aoqian Zhang, Jianmin Wang, and Philip S. Yu. 2015. SCREEN: Stream Data Cleaning under Speed Constraints. In SIGMOD. 827–841.

[14] David R. Brillinger. 2001. Time series - data analysis and theory. Classics in applied mathematics, Vol. 36. SIAM.

[15] Everette S Gardner Jr. 2006. Exponential smoothing: The state of the art—Part II. International journal of forecasting 22,4 (2006), 637–666.

[16] Rudolph Emil Kalman. 1960. A new approach to linear filtering and prediction problems. (1960)

[17] Kenji Yamanishi and Jun’ichi Takeuchi. 2002. A unifying framework for detecting outliers and change points from non-stationary time series data. In SIGKDD. ACM, 676–681.

[18] Xiaoyu Han, Haoran Xiong, Zhenying He, Peng Wang, Chen Wang, and X. Sean Wang. 2024. Akane: Perplexity-Guided Time Series Data Cleaning. Proc. ACM Manag. Data 2, 3 (2024), 121.

[19] Aoqian Zhang, Shaoxu Song, and Jianmin Wang. 2016. Sequential Data Cleaning: A Statistical Approach. In SIGMOD. 909–924

[20] Tian W, Shi J, Yiu M L. Efficient Methods for Accurate Sparse Trajectory Recovery and Map Matching[C]//2025 IEEE 41st International Conference on Data Engineering (ICDE). IEEE, 2025: 363-375.

[21] Aoqian Zhang, Zexue Wu, Yifeng Gong, Ye Yuan, and Guoren Wang. 2024. Multivariate Time Series Cleaning under Speed Constraints. Proc. ACM Manag. Data 2, 6 (2024), 245:1–245:26.

[22] Xiaoou Ding, Genglong Li, Hongzhi Wang, Chen Wang, and Yichen Song. 2024. Time Series Data Cleaning Under Expressive Constraints on Both Rows and Columns. In ICDE. 3682–3695.

[23] Fang Z, Shao W, Lu Z, et al. SHoTClean: Bridging Soft and Hard Constraints for Multivariate Time Series Cleaning[J]. Proceedings of the ACM on Management of Data, 2026, 4(1 (SIGMOD): 1-27.

[24] Liu Y, Zhao K, Cong G, et al. Online anomalous trajectory detection with deep generative sequence modeling[C]//2020 IEEE 36th International Conference on Data Engineering (ICDE). IEEE, 2020: 949-960.

[25] Shreshth Tuli, Giuliano Casale, and Nicholas R. Jennings. 2022. TranAD: Deep Transformer Networks for Anomaly Detection in Multivariate Time Series Data. Proc. VLDB Endow. 15, 6 (2022), 1201–1214.

[26] Yuhang Chen, Chaoyun Zhang, Minghua Ma, Yudong Liu, Ruomeng Ding, Bowen Li, Shilin He, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. 2023. ImDiffusion: Imputed Diffusion Models for Multivariate Time Series Anomaly Detection. Proc. VLDB Endow. 17, 3 (2023), 359–372.

[27] Douglas D H, Peucker T K. Algorithms for the reduction of the number of points required to represent a digitized line or its caricature[J]. Cartographica: the international journal for geographic information and geovisualization, 1973, 10(2): 112-122.

[28] Hershberger J E, Snoeyink J. Speeding up the Douglas-Peucker line-simplification algorithm[J]. 1992.

[29] Long C, Wong R C W, Jagadish H V. Direction-preserving trajectory simplification[J]. Proc. VLDB Endow., 2013, 6(10): 949-960.

[30] Muckell J, Hwang J H, Patil V, et al. SQUISH: an online approach for GPS trajectory compression[C]//Proceedings of the 2nd international conference on computing for geospatial research & applications. 2011: 1-8.

[31] Bellman R. On the approximation of curves by line segments using dynamic programming[J]. Communications of the ACM, 1961, 4(6): 284.

[32] Long C, Wong R C W, Jagadish H V. Trajectory simplification: On minimizing the direction-

based error[J]. Proc. VLDB Endow., 2014, 8(1): 49-60.

[33] Elman J L. Finding structure in time[J]. Cognitive science, 1990, 14(2): 179-211.

[34] Sutskever I, Vinyals O, Le Q V. Sequence to sequence learning with neural networks[J]. Advances in neural information processing systems, 2014, 27.

[35] Rumelhart D E, Hinton G E, Williams R J. Learning internal representations by error propagation[R]. 1985.

[36] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[J]. Advances in neural information processing systems, 2017, 30.

[37] Rákos O, Aradi S, Bécsi T, et al. Compression of vehicle trajectories with a variational autoencoder[J]. Applied Sciences, 2020, 10(19): 6739.

[38] Wang Z, Long C, Cong G. Trajectory simplification with reinforcement learning[C]//2021 IEEE 37th International Conference on Data Engineering (ICDE). IEEE, 2021: 684-695.

[39] Bellman R. A Markovian decision process[J]. Journal of mathematics and mechanics, 1957, 6(5): 679-684.

[40] Fang Z, He C, Chen L, et al. A lightweight framework for fast trajectory simplification[C]//2023 IEEE 39th International Conference on Data Engineering (ICDE). IEEE, 2023: 2386-2399.

[41] Dijkstra E W. A note on two problems in connexion with graphs[M]//Edsger Wybe Dijkstra: his life, work, and legacy. 2022: 287-290.

[42] Hart P E, Nilsson N J, Raphael B. A formal basis for the heuristic determination of minimum cost paths[J]. IEEE transactions on Systems Science and Cybernetics, 1968, 4(2): 100-107.

[43] Lou Y, Zhang C, Zheng Y, et al. Map-matching for low-sampling-rate GPS trajectories[C]//Proceedings of the 17th ACM SIGSPATIAL international conference on advances in geographic information systems. 2009: 352-361.

[44] Ren H, Ruan S, Li Y, et al. Mtrajrec: Map-constrained trajectory recovery via seq2seq multi-task learning[C]//Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining. 2021: 1410-1419.

[45] Shi Y, Gao H, Rao W. TIGAN: Trajectory imputation via generative adversarial network[C]//International Conference on Advanced Data Mining and Applications. Cham:

Springer Nature Switzerland, 2023: 195-209.

[46] Xu J, Li X, Lu W, et al. A trajectory-conditional generative adversarial network model for missing vehicle trajectory imputation[J]. Physica A: Statistical Mechanics and its Applications, 2025: 130881.

[47] Wei H, Chen C, Liu C, et al. Learning to simulate on sparse trajectory data[C]//Joint European Conference on Machine Learning and Knowledge Discovery in Databases. Cham: Springer International Publishing, 2020: 530-545.

[48] Chen Y, Zhang H, Sun W, et al. Rntrajrec: Road network enhanced trajectory recovery with spatial-temporal transformer[C]//2023 IEEE 39th International Conference on Data Engineering (ICDE). IEEE, 2023: 829-842.

[49] Wu D, Fang Z, Sun Q, et al. TrajRecovery: an efficient vehicle trajectory recovery framework based on urban-scale traffic camera records[C]//Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. 2024: 5979-5990.

参考答案

习题 3.1（轨迹大数据治理的核心任务）

参考答案：

轨迹大数据治理主要包括质量评估、轨迹清洗、轨迹压缩和轨迹恢复四类基础任务。

质量评估用于识别轨迹数据是否存在缺失、异常、漂移、采样不均或时空不连续等问题；轨迹清洗则针对这些问题进行修正、筛选或规范化处理，以提升数据可信度。轨迹压缩主要解决高频采样带来的冗余问题，其目标是在尽量保留轨迹主要形状、运动特征和语义信息的前提下，减少数据规模。轨迹恢复则主要面向轨迹缺失、采样稀疏和片段断裂等问题，通过插值、地图匹配或学习模型重建缺失内容。

从处理流程看，质量评估通常处于前置位置，为后续清洗、压缩和恢复提供依据。清洗关注“纠错”，压缩关注“减量”，恢复关注“补全”。在实际系统中，这几类任务也可能根据应用需求交叉配合使用。

习题 3.2（任务无关评估与任务感知评估）

参考答案：

任务无关的数据质量评估关注轨迹数据本身是否可靠，主要检查完整性、一致性、及时

性、有效性等基础质量问题。例如，是否存在坐标越界、时间乱序、字段缺失或轨迹断裂等现象。它回答的是“数据本身质量如何”。

任务感知的数据质量评估则进一步考虑这些质量问题会对具体应用造成多大影响。相同的数据缺陷，在不同任务中的重要性并不一致。例如，短时间轨迹缺失对起点统计影响可能较小，但对轨迹预测、驾驶行为识别等任务却可能显著破坏时序模式。

因此，仅依赖任务无关评估是不够的。实际治理中，应先通过任务无关评估掌握数据基础质量，再结合具体任务判断哪些问题最值得优先修复，从而平衡清洗成本与任务收益。

习题 3.3（T-Assess 轨迹专用质量评估框架）

参考答案：

T-Assess 之所以更适合轨迹数据，是因为它不仅关注一般意义上的缺失值、重复值和异常值，还专门考虑了轨迹数据的时空连续性、运动规律、道路网络约束以及时空分布偏差。相比 TsQuality、TFDV 和 F4U 等通用方法，它更贴近轨迹数据治理的实际需求。

T-Assess 从有效性、完整性、一致性和公平性四个维度构建质量评估体系。有效性关注时间戳、经纬度等基础字段是否满足合法约束；完整性关注轨迹是否存在缺失点、断裂和字段缺项；一致性关注轨迹是否符合合理的运动规律、平滑性和道路拓扑约束；公平性则考察轨迹数据在不同时空区域上的覆盖是否均衡。

公平性维度具有重要意义。若城市中心区域数据密集而郊区数据稀疏，或白天数据充分而夜间数据不足，下游模型就可能在数据稀缺区域表现较差。因此，公平性评价能够帮助识别数据分布偏差，提升后续分析和预测任务的可靠性。

习题 3.4（轨迹异常的判定标准）

参考答案：

从物理约束角度看，若轨迹点之间的速度、加速度、位移变化明显超过移动对象的合理运动能力，则可判定为异常。例如，车辆在极短时间内跨越数十公里，显然违背运动规律。

从统计离群角度看，若某个轨迹点明显偏离局部轨迹趋势或整体分布模式，也可能属于异常。例如，一条整体平稳前进的行人轨迹中突然出现一个远离主路径的孤立点，随后又返回原路径，这通常说明存在定位漂移。

从地图匹配角度看，车辆观测点远离候选道路或片段不满足道路连通性，可能提示定位错误。例如，普通道路车辆短时间内从道路跳到远处水域，应进一步核查。判断时需考虑桥梁、停车场、定位误差及路网缺失，不能把所有地图偏离都作为脏点删除。

习题 3.5（SHoTClean 的硬约束与软约束）

参考答案：

SHoTClean 的硬约束限定点对之间允许的变化率或速度范围，用于筛除不可行连接；软约束根据观测对先验分布的偏离程度评分，并结合时间衰减选择路径。它原本面向多变量时间序列，用于轨迹时还需确定坐标表示、变量尺度及约束含义。

在 SHoTClean 中，每个轨迹点可看作图中的节点，若较早时刻的点到较晚时刻的点满足硬约束，则允许建立有向连接；连接关系或节点本身再根据软约束赋予权重。这样，轨迹清洗问题便可转化为在有向无环图中寻找一条最大权重路径的问题。

最大权重路径被视为最可信的轨迹主干，未被选中的点则可能是异常点。与单纯逐点检测相比，这种方法能够从整条序列的整体一致性出发做判断，可结合约束与统计得分筛查异常，但不保证未入选点都错误，也不保证修改点数最少。

习题 3.6（TranAD 在轨迹清洗中的作用）

参考答案：

TranAD 利用 Transformer 结构对多变量时间序列进行重构，并以重构误差识别异常。若某个时间点无法被模型较好重构，说明它与正常时序模式存在较大偏差，因此可能是异常点。

其两阶段重构机制是该方法的重要特点。第一阶段先完成初步重构并计算误差信息；第二阶段根据这些误差信息进一步聚焦潜在异常位置，从而放大正常区域与异常区域之间的差异，提高异常检测能力。

TranAD 可联合建模多变量的时序关系，其注意力结构有助于利用历史上下文。迁移到轨迹数据时，应先构造位置、速度等特征，处理不规则采样，并结合地图和运动约束验证异常候选。原论文并未据此保证长轨迹清洗效果，检测之后还需单独设计修复步骤。

习题 3.7（传统轨迹压缩方法对比）

参考答案：

Douglas-Peucker 算法采用递归分裂思想，以轨迹点到起终点连线的最大偏差为依据保留关键点，适合离线几何压缩。它结构简单，但主要关注空间误差，忽略了时间因素。

SQUISH 面向流式场景，通过维护固定容量缓冲区，动态删除当前最不重要的点，适合实时轨迹传输和在线压缩。其优势是能够边接收边压缩，但压缩效果较依赖缓冲区大小，局部决策可能带来全局误差。

Error-Search 则强调在给定存储预算下优化整体方向误差，能够获得较高质量的压缩结

果，适合对效果要求较高的离线任务。不过其全局搜索代价较高，不适合大规模实时应用。

习题 3.8（深度学习轨迹压缩）

参考答案：

RLTS 将轨迹压缩建模为马尔可夫决策过程，通过强化学习决定哪些点应被保留或删除。模型在压缩率和误差控制之间学习删除策略，因此本质上是一种面向序列决策的压缩方法。

S3 则将压缩问题建模为自监督的“压缩—重构”过程。其核心思想是：如果某一组关键点能够支持模型较好地重构原始轨迹，那么这些点就具有较高代表性，应当被保留。因此，它更强调通过重构能力学习关键点选择机制。

二者的主要区别在于，RLTS 关注动态决策与策略优化，而 S3 关注表示学习与自监督重构。前者更像“逐步做选择”，后者更像“通过能否重建来判断哪些点重要”。

习题 3.9（轨迹恢复方法比较）

参考答案：

线性插值假设相邻观测点之间的运动近似为匀速直线，因此实现简单、效率很高，适合采样间隔较短且运动变化平缓的场景。但它忽略道路结构和复杂运动规律，可能生成不符合实际地理环境的恢复轨迹。

ST-Matching 将空间距离、道路拓扑、时间约束和速度合理性结合起来，在候选路径中选择最可能的轨迹恢复结果，更适用于低采样率车辆轨迹。它能够生成更符合道路网络的恢复路径，但也更依赖路网质量和候选路径搜索效率。

MTrajRec 则采用深度学习的端到端恢复思路，将低采样轨迹直接映射为高采样率、受路网约束的轨迹序列。它能够学习复杂时空模式，可减轻“先插值、再匹配”的误差传递，仍可能出现道路预测错误，但需要较大规模训练数据和更高模型开销。